



**DALHOUSIE
UNIVERSITY**

CSCI 5410

Serverless Data Processing

Sprint 3 Report

Team Members

Kaushik Chanabhai Dhola (B00956419)

Luv Prakashkumar Patel (B00950942)

Jaishankar Mohanraj(B00975938)

MD Samshad Rahman (B00968344)

Gitlab Repo link : https://git.cs.dal.ca/mohanraj/serverless_group1

Deployed URL : <https://dalvac-qual5imuuq-uc.a.run.app/>

DALHOUSIE UNIVERSITY

HALIFAX

Table of contents

Table of contents	2
Table of figures:	3
Virtual Assistant Module - [Kaushik Chanabhai Dhola]:	6
Admin Dashboard Testing:	15
User Authentication [Luv Patel]	19
General Tasks [Jaishankar Mohanraj]:	36
Frontend template and setup:	36
SQS [Jaishankar Mohanraj]:	37
Notifications [Jaishankar Mohanraj]:	38
Message Passing Module [Jaishankar Mohanraj]:	39
Individual contribution [Jaishankar Mohanraj]:	41
Data Analysis and Visualization: [MD Samshad Rahman].	47
Test cases:	49
Test cases:	52
Deployment:	53
IaC:	53
Meeting Logs:	56
References:	58

Table of figures:

Figure 1 Virtual Assistant Issue- Git board	7
Figure 2 Admin Dashboard Issue- Git board	8
Figure 3 Chatbot Architecture	8
Figure 4 bookRoom Lex Intent Workflow	9
Figure 5 RegisterGuide Lex Intent Workflow.....	9
Figure 6 Greeting Lex Intent Workflow.....	9
Figure 7 Talk to Agent Intent	9
Figure 8 Navigation Booking Intent	10
Figure 9 Show Available Rooms Intent	10
Figure 10 Feature Navigation Intent	10
Figure 11 Test 1 Successful Retrieval of booking Information.....	11
Figure 12 Test 1 Successful Retrieval of booking Information 1	11
Figure 13 Test 2 Invalid Response from Chatbot.....	12
Figure 14 Test 3 User Guide Chat.....	12
Figure 15 Test 3 User Guide Chat 1	13
Figure 16 Test 3 User Guide Chat 2.....	13
Figure 17 Test 3 User Guide Chat 3.....	14
Figure 18 Test 3 User Guide Chat 4.....	14
Figure 19 Session wise chat history	15
Figure 20 Admin Add Room Test Case 1	16
Figure 21 Admin Add Room Test Case 2.....	16
Figure 22 Added in the DynamoDB	16
Figure 23 Admin Update Room Test 1	17
Figure 24 Updated in DynamoDB	17
Figure 25 Admin Delete Room Test 1	18
Figure 26 Admin Delete Room Test 2.....	18
Figure 27 Delete record from DynamoDB	18
Figure 28 Updated Record in the Update Form.....	18
Figure 29 Issue created in the Issueboard	21
Figure 30 Issue Created in the issueboard	21
Figure 31: Algorithm for signin module authentication.....	22

Figure 32: Algorithm for user group assignment	22
Figure 33: Authentication Architecture Diagram	23
Figure 34: Homepage when user enters the website	23
Figure 35: Application Sign in Page	24
Figure 36: User enters incorrect credentials	24
Figure 37: Entering correct credentials	25
Figure 38: Security Answer Page	25
Figure 39: Decrypt Cipher text	26
Figure 40: Entering the correct text	26
Figure 41: Homepage with signout component	27
Figure 42: Cognito SignUp page with valid credentials	27
Figure 43: User account verification code page	28
Figure 44: Verification code mail to user	28
Figure 45: Enter the correct verification code	29
Figure 46: User security Question Setup page	29
Figure 47: User sign in prompt page	30
Figure 48: User security answer page	30
Figure 49: User not satisfying the password requirement	31
Figure 50: User enters wrong verification code	31
Figure 51: User enters incorrect decrypt code	32
Figure 52: User signup with already registered account	32
Figure 53: Admin dashboard sidebar navigation	33
Figure 54: User tries to navigate manually without signin	33
Figure 55: User signup with already registered account	34
Figure 56: User tries to navigate manually without signin	34
Figure 57: User tries to navigate manually without signin	35
Figure 58: User entries when the user creates and account using MFA	35
Figure 59: User login timestamp gets recorded every time they login to the application	36
Figure 60: backend architecture for bookings	38
Figure 61: SNS Architecture	39
Figure 62: AWS part of the backend for live chat	40
Figure 63: GCP part of the backend for live chat	41
Figure 64: Landing page of the website	41

Figure 65: Features of the resort	42
Figure 66: view rooms page.....	42
Figure 67: Booking page.....	42
Figure 68: booking confirmation email.	43
Figure 69: Booking unavailable email.	43
Figure 70: Agent side active conversation list	43
Figure 71: Chat Page.....	44
Figure 72: End conversation button	44
Figure 73: Conversation has already been closed message	44
Figure 74: Email sent to user with link for live chat with agent.	44
Figure 75: No available agents now message	45
Figure 76: login notifications.....	45
Figure 77: Booking issue.	46
Figure 78: Chat module.....	46
Figure 79: Notifications	46
Figure 80: System architecture for feedback	48
Figure 81 All existing feedbacks with polarity	49
Figure 82 Logged-in user's own feedbacks with delete button.....	50
Figure 83 Submit feedback	50
Figure 84 Delete feedback	51
Figure 85 System Architecture for Data visualization (Admin Dashboard).....	51
Figure 86 Admin Dashboard	52
Figure 87 Manual lambda trigger to update CSV	52
Figure 88 Cron job expression	53
Figure 89 Lambda Invocation every 5 minutes	53
Figure 90: Terraform IaC for frontend	54
Figure 91: Frontend has been deployed using google cloud run	54
Figure 92: terraform script to create new deployment.....	55
Figure 93: Dockerfile to create image.....	55

Virtual Assistant Module - [Kaushik Chanabhai Dhola]:

Research Details:

Introduction to usage of Virtual Assistant:

1. **Objective:** The objective of the virtual assistant (DalBot) is to assist users in retrieving booking details and providing guidance on using the application effectively.
2. **Market Research:** I have conducted research to understand user needs and preferences in interacting with virtual assistants for booking services and application guidance. It incorporates how the existing Assistants are responding based on the requests and question provided.

To design and develop an effective user interface (UI) and ensure user experience (UX) for DalBot, I have conducted proper market research, particularly focusing on industry leaders like Expedia. The research aimed to understand best practices in chatbot interaction and scalability in handling a large user base. Specifically, insights were gathered from [Expedia's Help Center](#) [1] regarding:

- **UI/UX Design:** Analyzed Expedia's UI/UX strategies to optimize user engagement and navigation.
- **Chatbot Interaction:** Studied the mechanisms incorporated by Expedia to interact with users through chatbots.
- **Scalability:** Explored strategies used by Expedia to manage and scale chatbot interactions across a diverse user demographic.

This research has been essential in developing the Virtual assistant for our system as it provided proper insights as well as the workflow that the global leaders have incorporated.

3. Technology Stack:

- a. **Amazon Lex:** Used for natural language understanding and conversation management. This Amazon services automatically processes the request message and extract the meaning using which the specified intent would be triggered to perform the operation.
- b. **AWS Lambda:** The lambda function is used to serves as a serverless compute service for executing backend logic. Right now, I am retrieving the booking details based on the details provided by the user.
- c. **DynamoDB:** This includes the booking details and user information. This is from where lambda function gets data to present to users on chatbot.
- d. **React:** This is the JavaScript library used for building the User Interface for the Chatbot. I have not used any pre-built libraries.
- e. **JavaScript, HTML, CSS:** Some frontend technologies used for developing the user interface.

Individual Contribution [Kaushik Chanabhai Dhola]:

My responsibilities:

1. I am responsible for development and integration Amazon Lex with Lambda function and DynamoDB frontend, and coordinating Front-end Chatbot UI which is implemented with React, JavaScript, HTML, CSS.
2. I have also implemented the admin dashboard with frontend and backend functionalities:
 - a. Add New Room: This form adds new type of room based on the capacity, availability, type, features and cost.
 - b. Delete Room: This form deletes room from based on the combination of type and capacity given.
 - c. Update Room Details: This function enables user to update the existing room details such as feature, cost, availability based on the combination of type and cost given.

My tasks:

- I have implemented Amazon Lex integration with a Lambda function to facilitate interaction with DynamoDB for booking details retrieval.
- I have also developed the frontend chatbot UI using Kommunicate AI powered platform ensuring full functionality. This functionality supports features such as state management and Interaction History Archive.
- I have developed frontend for admin dashboard using React, HTML/CSS, JavaScript.
- I have implemented the frontend for the admin dashboard for Add Room Details, Update Room Details, Delete Room using for Lambda Functions AddNewRoom, UpdateRoomDetails, DeleteRoom and FetchRoomDetails. These all functions were implemented using python 3.12 and by attaching library layers to the lambda functions.
- I have worked on the deployment process with Jaishankar to deploy the project. Also worked on the Terraform (IaC with Jaishankar) for the automated provisioning of deployment resources.

As shown below, git issue has the given 3 tasks have been implemented and closed successfully.

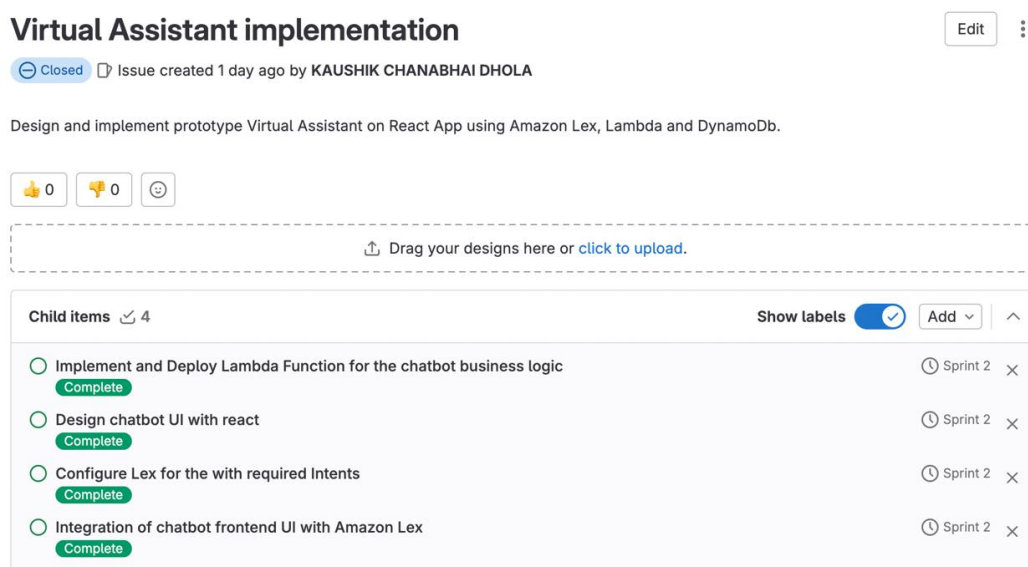


Figure 1 Virtual Assistant Issue- Git board

Property Agent Portal Implementation

Edit ⋮

Closed Issue created 2 weeks ago by Luv Prakashkumar Patel

Property agent view for the website where they can check room bookings, edit rooms, delete rooms, as well as interact with the customer over the chat feature.

0 0

Drag your designs here or [click to upload](#).

Child items 3 **Show labels** Add

Admin Add New Room
Complete

Admin Delete Room
Complete

Admin Update Room
Complete

Sprint 3

Sprint 3

Sprint 3

Figure 2 Admin Dashboard Issue- Git board

Architecture Design:

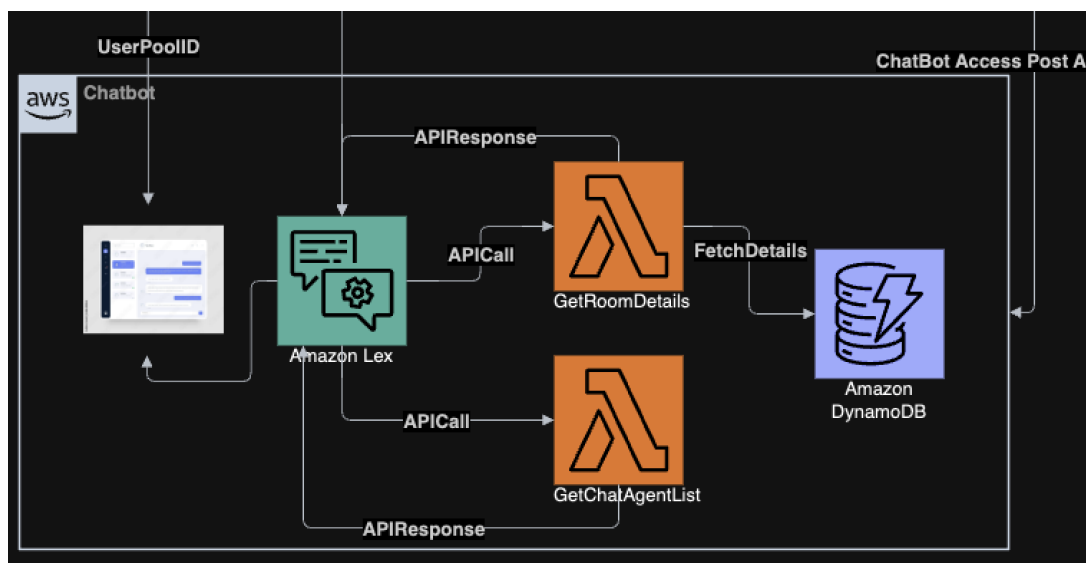


Figure 3 Chatbot Architecture

Pseudocode/ Algorithm for important modules' logic:

Lex DalBot Intents Flow:

1. bookRoom Intent:

Below figure shows the dynamic workflow of the intent for the DalBot. In this workflow, you can see the logic behind how the request is handled and how the response is generated for the bot. The request is first forwarded to lambda function which will process the query of request and generates the response based on the business logic applied on the data passed in event and data store in the DynamoDB.

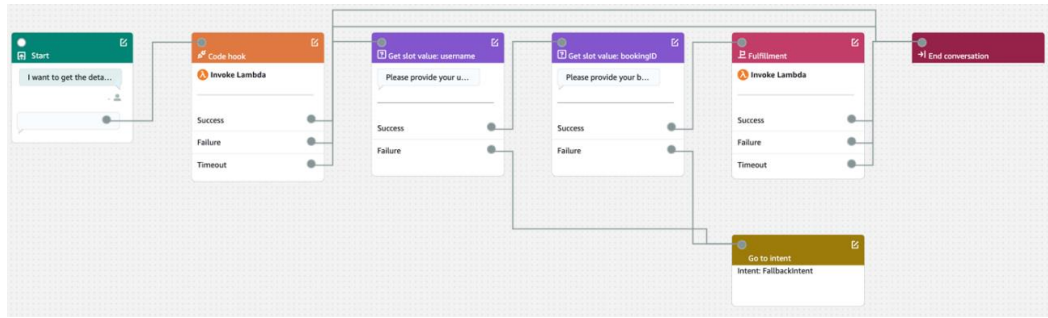


Figure 4 bookRoom Lex Intent Workflow

2. RegisterGuide Intent:

The below given workflow for the register guide is quite naïve as it required static response to be generated.

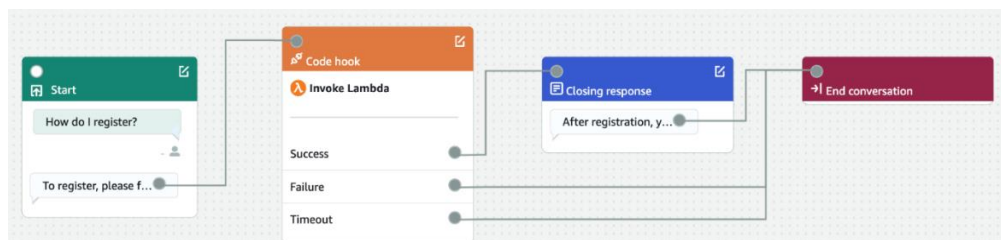


Figure 5 RegisterGuide Lex Intent Workflow

3. Greeting Intent:

The below given workflow for the Greetings to the user.

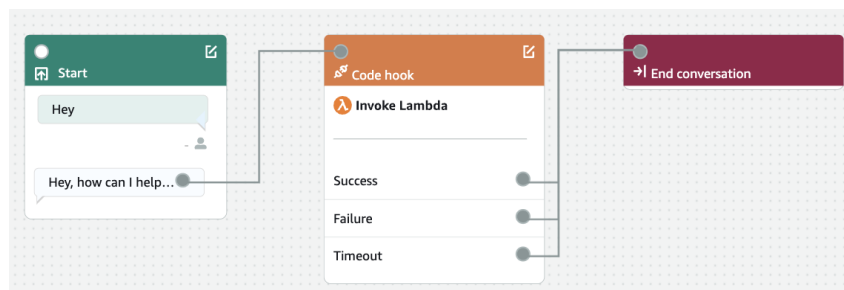


Figure 6 Greeting Lex Intent Workflow

4. TalkToAgent Intent:

The below given workflow is for the redirecting the user to the live chat conversation page.

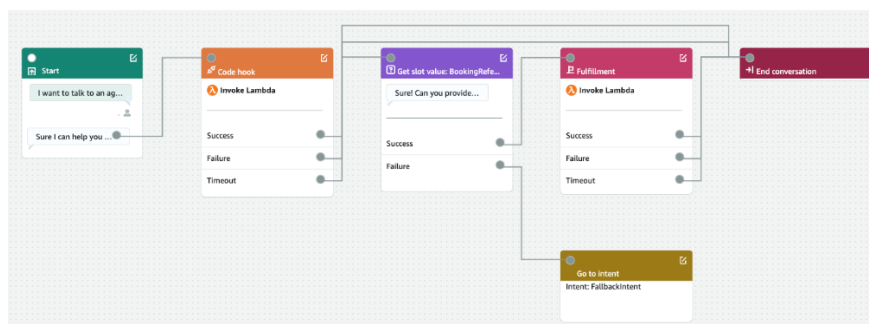


Figure 7 Talk to Agent Intent

5. Navigation Booking Intent:

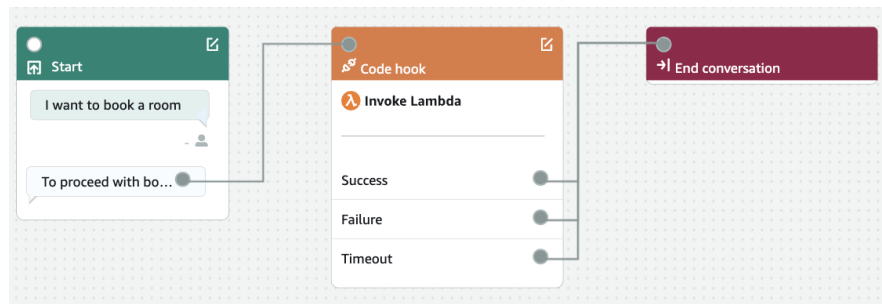


Figure 8 Navigation Booking Intent

6. Show Available Rooms Intent:

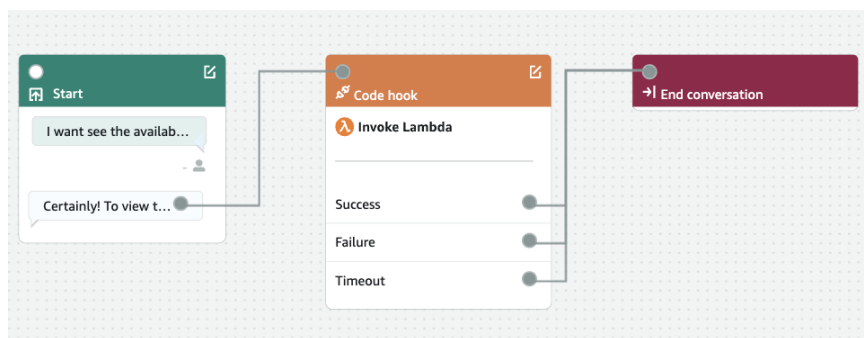


Figure 9 Show Available Rooms Intent

7. Feature Navigation Intent:

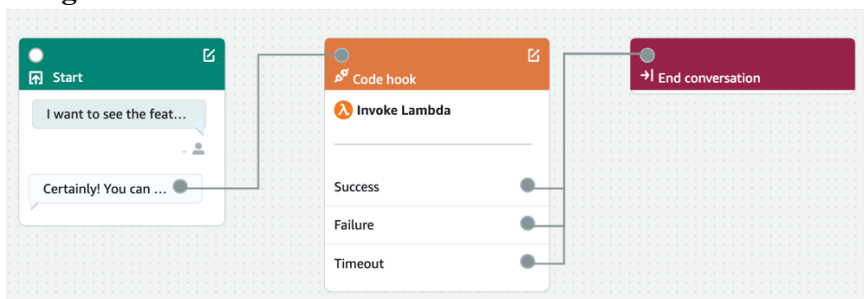


Figure 10 Feature Navigation Intent

Test Cases for Virtual Assistant and Admin Dashboard:

The implemented solution showcases a fully integrated chatbot UI with Amazon Lex, achieving end-to-end functionality. This integration leverages Kommunicate AI to facilitate the front-end and serverless interactions between Amazon Lex and AWS services such as Lambda and DynamoDB.

Successful Retrieval of booking information:

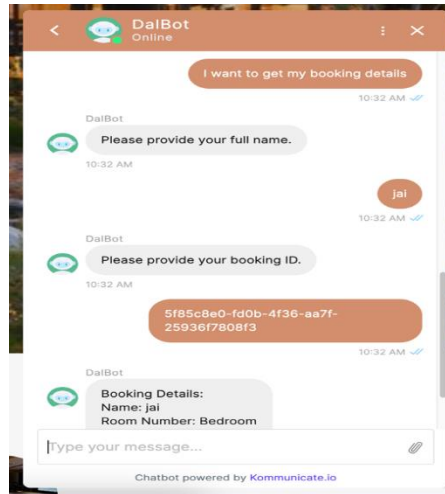


Figure 11 Test 1 Successful Retrieval of booking Information

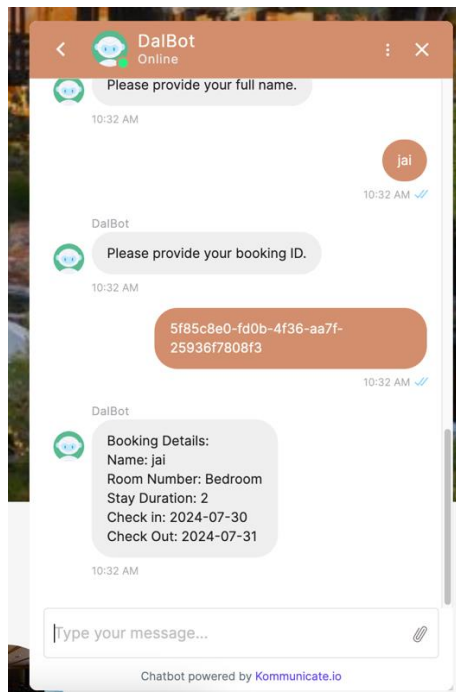


Figure 12 Test 1 Successful Retrieval of booking Information 1

Whenever user request to get the booking information from the bot, it will first check if the user exists in the system, followed by the booking ID reference check. If these both checks are valid then only the booking details can be retrieved from the DynamoDB.

If the details are not valid then the below response would be generated from the lambda function:

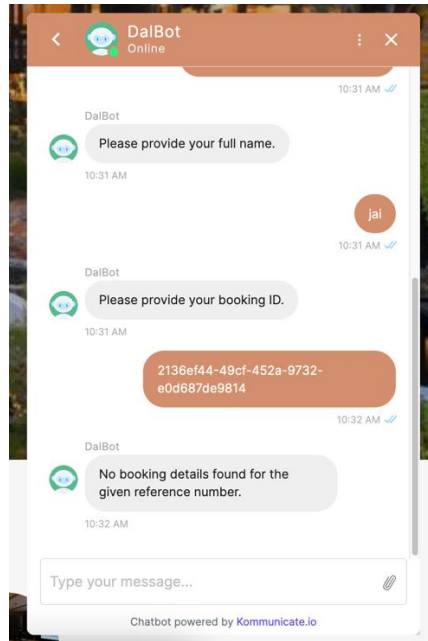


Figure 13 Test 2 Invalid Response from Chatbot

User Guide Chat:

If the user wants to know the details on web application FAQs or information regarding application access, then they can easily navigate to the chatbot and ask whatever question they might have in their mind. Below is the scenario where user asks, “How to register for a account” and gets response on the procedure for the registration.

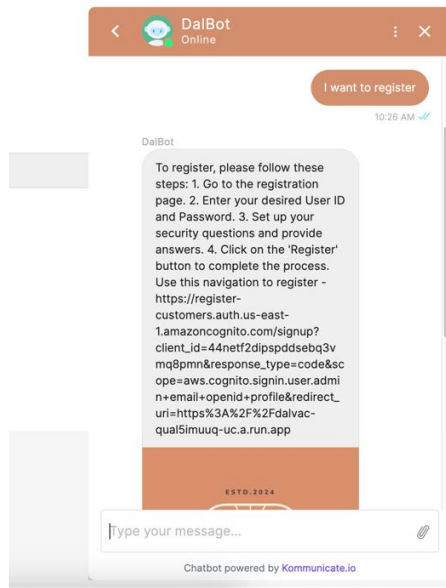


Figure 14 Test 3 User Guide Chat

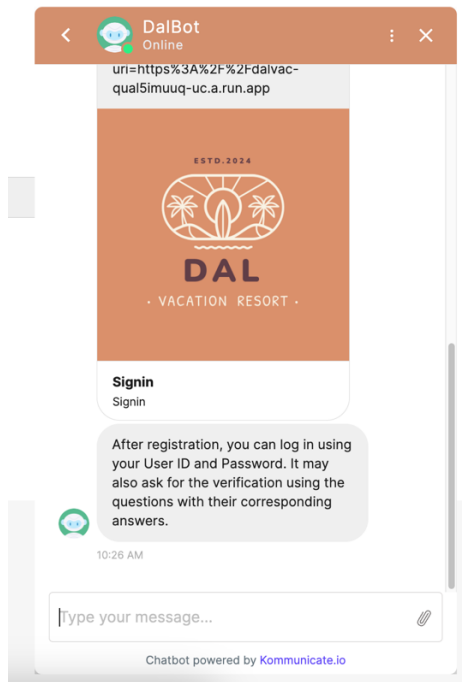


Figure 15 Test 3 User Guide Chat 1

Available Room Guide:

If user wants to inquire about the available rooms, then they can directly ask a query in the chatbot which will provide a user guide to know about the available rooms.

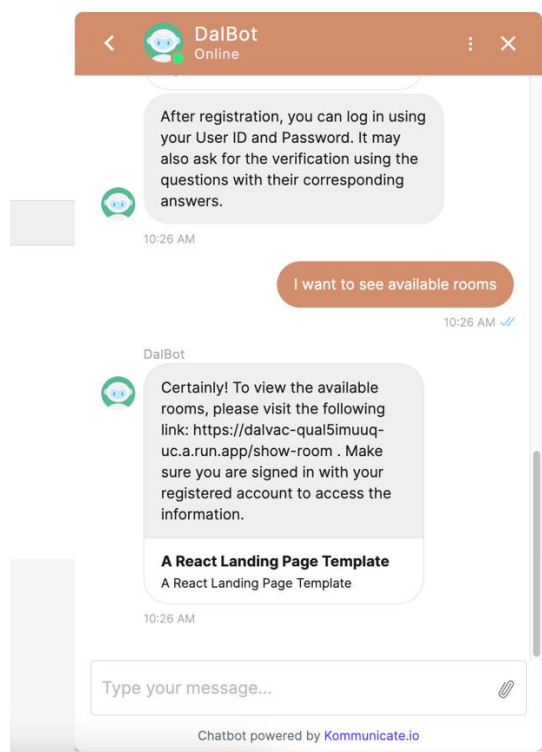


Figure 16 Test 3 User Guide Chat 2

Book a Room:

If user wants to know about the booking process, then they can directly ask a query in the chatbot which will provide a user guide to know about the book room procedure.

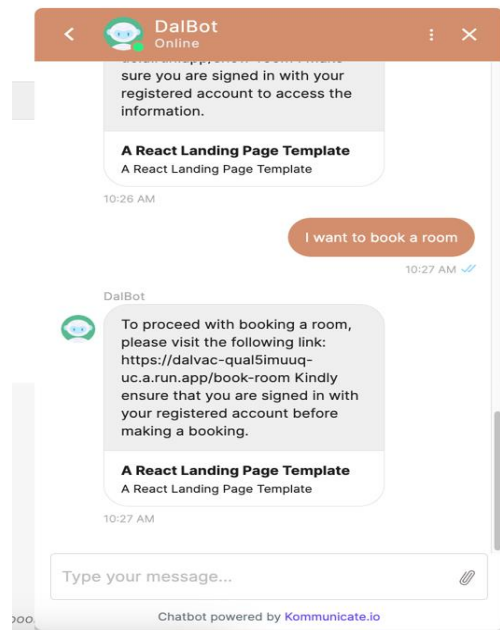


Figure 17 Test 3 User Guide Chat 3.

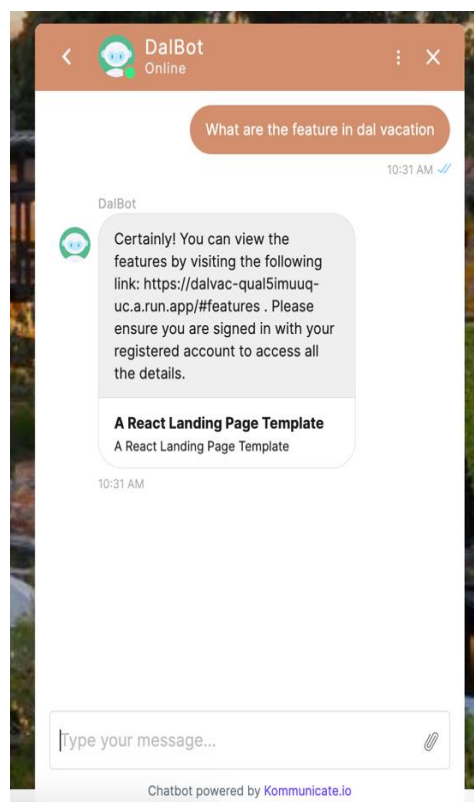


Figure 18 Test 3 User Guide Chat 4

Session wise chat history:

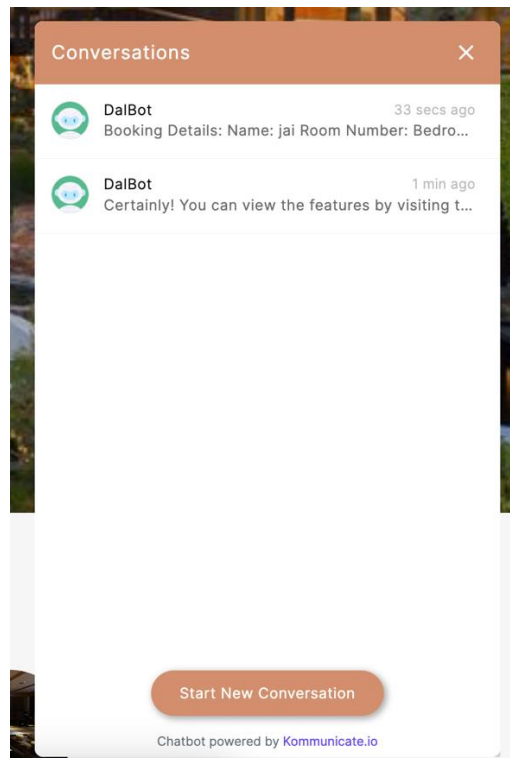


Figure 19 Session wise chat history

Admin Dashboard Testing:

Add Room Testing:

Here, the below page is for adding new room with the type, capacity, feature, available rooms and cost. It is only accessed by the admin.

ADD NEW ROOM

Please fill out the form below to add a new room.

Room Type:

Capacity:

Features:

Available Rooms:

Cost:

ADD ROOM

Figure 20 Admin Add Room Test Case 1

In the below snapshot, you can see that I can add the product using the given form.

ADD NEW ROOM

Please fill out the form below to add a new room.

Room Type:

Capacity:

Features:

Available Rooms:

Cost:

ADD ROOM

Room added successfully!

Figure 21 Admin Add Room Test Case 2

The below given screenshot shows the data being added in the DynamoDB.

rooms Autopreview View table details

► **Scan or query items**
Expand to query or scan items.

Completed. Read capacity units consumed: 2

Items returned (2) Actions Create item

Type (String)	Capacity (String)	Available Rooms	Cost	Features
Bedroom	2	2	35	amazing
Conference Room	30	2	100	With large space available to host an event

Figure 22 Added in the DynamoDB

Update Room Testing:

Update Room Quantity

Welcome to the admin page. Please fill out the form below to update room quantities.

Admin

Bedroom
✓ Conference Room

Capacity: 30

Features: With large space available to host an event

Available Rooms: 2

Cost: 150

Submit

Figure 23 Admin Update Room Test 1

Update Room Quantity

Welcome to the admin page. Please fill out the form below to update room quantities.

Admin

Room Type: Conference Room

Capacity: 30

Features: With large space available to host an event, and this is updated feature

Available Rooms: 2

Cost: 100

Submit

Figure 22 Admin Update Room Test 2

The below given screenshot shows the existing data being updated in the DynamoDB.

rooms Autopreview View table details

Scan or query items
Expand to query or scan items.

Completed. Read capacity units consumed: 2

Items returned (2)

	Type (String)	Capacity (String)	Available Rooms	Cost	Features
☐	Bedroom	2	2	35	amazing
☐	Conference Room	30	2	100	With large space available to host an event, and this is up...

Figure 24 Updated in DynamoDB

Admin Delete Room Testing:

Admin Menu

- Add Room
- Update Room
- Delete Room
- Statistics
- Conversations
- Sign Out

Delete Room

Please select the room type and capacity to delete the room record.

Room Type: Conference Room

Capacity: 30

Delete

Figure 25 Admin Delete Room Test 1

Admin Menu

- Add Room
- Update Room
- Delete Room
- Statistics
- Conversations
- Sign Out

Delete Room

Please select the room type and capacity to delete the room record.

Room Type: Conference Room

Capacity: 30

Delete

Room deleted successfully

Figure 26 Admin Delete Room Test 2

The below given screenshot shows the data being deleted in the DynamoDB.

Scan or query items
Expand to query or scan items.

Completed. Read capacity units consumed: 2

Items returned (1)

	Type (String)	Capacity (String)	Available Rooms	Cost	Features
☐	Bedroom	2	2	35	amazing

Figure 27 Delete record from DynamoDB

Admin Menu

- Add Room
- Update Room
- Delete Room
- Statistics
- Conversations
- Sign Out

Update Room Quantity

Welcome to the admin page. Please fill out the form below to update room quantities.

Admin

Room Type: Bedroom

Capacity: 2

Features: amazing

Available Rooms: 2

Cost: 35

Submit

Figure 28 Updated Record in the Update Form

User Authentication [Luv Patel]

Research Details:

Introduction to User Authentication :

1. **Objective :** The objective of the user authentication is to allow only the authenticated users to access the features which are not available to the guest users who are just browsing the website. The goal is to implement a 3-layer authentication system to verify the users.
2. **Market Research :** I have done research on how to leverage the power of the AWS Cognito to perform the first layer user authentication. The second layer required the security question and answer page where the user needs to enter his/her security answer for the question which they set during registering. Next the 3rd layer is the Caesar cipher authentication step, where the user needs to decrypt the cipher that is provided to them using a decrypt key value.

Once, all these steps are successfully completed, the user is fully authenticated and can now access all the features of the website. To design and develop the required task, I have thoroughly gone through the secure government website like Canada Revenue Agency (CRA)[2] who already has this kind of multi-layer authentication.

- Examined CRA's flow for authentication and researched on how similar things can be done in our project.
- Learned the architecture to implement the required task while focusing on increasing the user engagement and experience.

This exploration played a crucial role in helping me understand the multi-factor authentication flow for users. I also got to know how it is currently being implemented by one of the government bodies where security and privacy are crucial aspects.

Following the documentation of the AWS for Cognito, I came across the user groups that I can create, so that I could manage the permissions of each user group separately, by giving specific access to each one. Moreover, I discovered that I need to configure the callback URLs for the sign-in and sign-up processes. These URLs should match with the URLs mentioned in the redirect URL that is present in the query string of the calling string. If these URLs are different, then the Cognito will not allow the redirection, thus stopping the program execution and the flow[4].

Technology Stack :

- a. **AWS Cognito** : Used to handle the authentication flow, specifically for the 1st layer authentication which is taken care by cognito. This service by AWS manages the user pool, user groups, and tokens of the user for seamless authentication and management.
- b. **AWS Lambda** : The lambda functions are used to implement the business logic by running the serverless compute service which handles the backend code, without having to worry about the server.
- c. **DynamoDB** : This is the database used for storing the information like the user security Q&A , as well as other information like the userRole, etc. This all functionality is achieved by the lambda functions which triggers the storage of data from the frontend to the database.
- d. **React** : It is a frontend framework based on JavaScript, used for development of the frontend for the DalVacationHome.

Individual Contribution [Luv Patel]

My responsibilities: I was assigned the task of implementing the multi-layer user authentication and integrating it with front-end technology like React which will also be developed by me. Moreover, I was responsible for integrating the backend lambda functions and database with the frontend to offer a complete experience to the users of the website.

My Task:

- I have implemented the 3 layer user authentication for the DalVacationHome web application using the AWS services like AWS Cognito, AWS Lambda as well as AWS DynamoDB.
- Development of the frontend part for integrating with the backend to offer a seamless experience to the users using out website.
- Creating user pool for the implementation so that users can authenticate themselves.
- Implementing lambda functions to interact with the dynamoDB and Cognito.
- Moreover, I have implemented the protected routes that restricts the users from accessing any page that they are restricted to or are not allowed unless they are authenticated to do so.
- Also, on the admin side, i have implemented teh sidebar navigation, that enables the admin to navigate between various components on the admin dashboard

Below shown screenshot shows the issue created on gitlab for project management as well as the corresponding tasks.

User Authentication

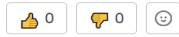
[Edit](#)

Closed Issue created 13 hours ago by Luv Prakashkumar Patel

Implement User Signup and Login with 3 Factor Authentication Implementation. The 3 layers are as follows:

- 1st Factor is user signup and login using AWS Cognito.
- 2nd Factor is security Question and Answer using AWS lambda and DyanmoDB.
- 3rd Factor is ceaser Cipher where user needs to decrypt the provided cipher using a key.
- Frontend development using React.

Edited 13 hours ago by Luv Prakashkumar Patel



Drag your designs here or [click to upload](#).

Child items 5

Show labels Add

Create User Pool for different users

Complete

Sprint 2 X

3 Level User Authentication

Complete

Sprint 2 X

Ceaser Cipher Implementation for 3rd factor authentication

Complete

Sprint 2 X

Design MFA Authentication UI with React

Complete

Sprint 2 X

Implement and deploy Lambda functions and Database for required Logic

Complete

Sprint 2 X

Figure 29 Issue created in the Issueboard

Jaishankar Mohanraj / CSCI5410-S24-SDP-Team_1 / Issues / #28

Closed **Property Agent Portal Implementation**

Child items 5

Show labels Add

Admin Add New Room

Complete

Sprint 3 X

Admin Delete Room

Complete

Sprint 3 X

Admin Update Room

Complete

Sprint 3 X

Admin Sidebar Navigation

Complete

Sprint 3 X

Protected Routes

Complete

Sprint 3 X

Linked items 0

Add

Link issues together to show that they're related or that one is blocking others. [Learn more](#).

Activity

Sort or filter

Add a to do

2 Assignees

Edit

Luv Prakashkumar Patel

KAUSHIK CHANABHAI DHOLA

Labels

Edit

Closed

Milestone

Edit

Sprint 3

Weight

Edit

None

Due date

Edit

None

Time tracking

+

No estimate or time spent

Health status

Figure 30 Issue Created in the issueboard

Algorithm for important module Logic:

1). Signin Module:

```
if (code) {
  try {
    const response = await axios.post('https://register-customers.auth.us-east-1.amazoncognito.com/oauth2/token'
    new URLSearchParams({
      grant_type: 'authorization_code',
      client_id: '44netf2dipsddsebq3vmq8pmn',
      code: code,
      redirect_uri: 'http://localhost:3000/signin' // The redirect URI
    })), {
      headers: {
        'Content-Type': 'application/x-www-form-urlencoded'
      }
    });
```

Figure 31: Algorithm for signin module authentication

=> The above displayed piece of code I believe, is the most important aspect of the project among many others, this is because this code is handling the authentication of the code that is returned by the AWS cognito after the user is successfully authenticated. Ideally, when the user authenticates themselves using cognito, it returns the users to the redirect URLs, along with a authentication code as a query string parameter, that helps identify the user authentication in the project frontend.

The above mentioned code takes in account the oauth2 link and calls with while sending the grant type, the code generated after the authentication is successful by the cognito, and the client ID for which app client this step needs to take place. This is where the exchange of the authentication tokens takes place. Without verification and exchange of the authentication tokens, we cannot move forward with the other tasks that will happen in the project .

2). User pool groups assigning :

```
def add_user_to_group(user_id, user_role):
    try:
        group_name = 'RegisteredUsers' if user_role == 'RegisteredUser' else 'PropertyAgents'

        response = cognito.admin_add_user_to_group(
            UserPoolId='us-east-1_uBDCBcAXG',
            Username=user_id,
            GroupName=group_name
        )
        print(f'User added to group {group_name} in Cognito User Pool')
    except Exception as e:
        print(f'Error adding user to group in Cognito User Pool: {str(e)}')
        raise
```

Figure 32: Algorithm for user group assignment

This is another important code, where the users are categorized in the groups as per the custom attributes in the cognito pool. This is where the users will be added to the respective group which will have different setting and permission on what they can perform in the websites and what they can access.

Architecture Diagram :

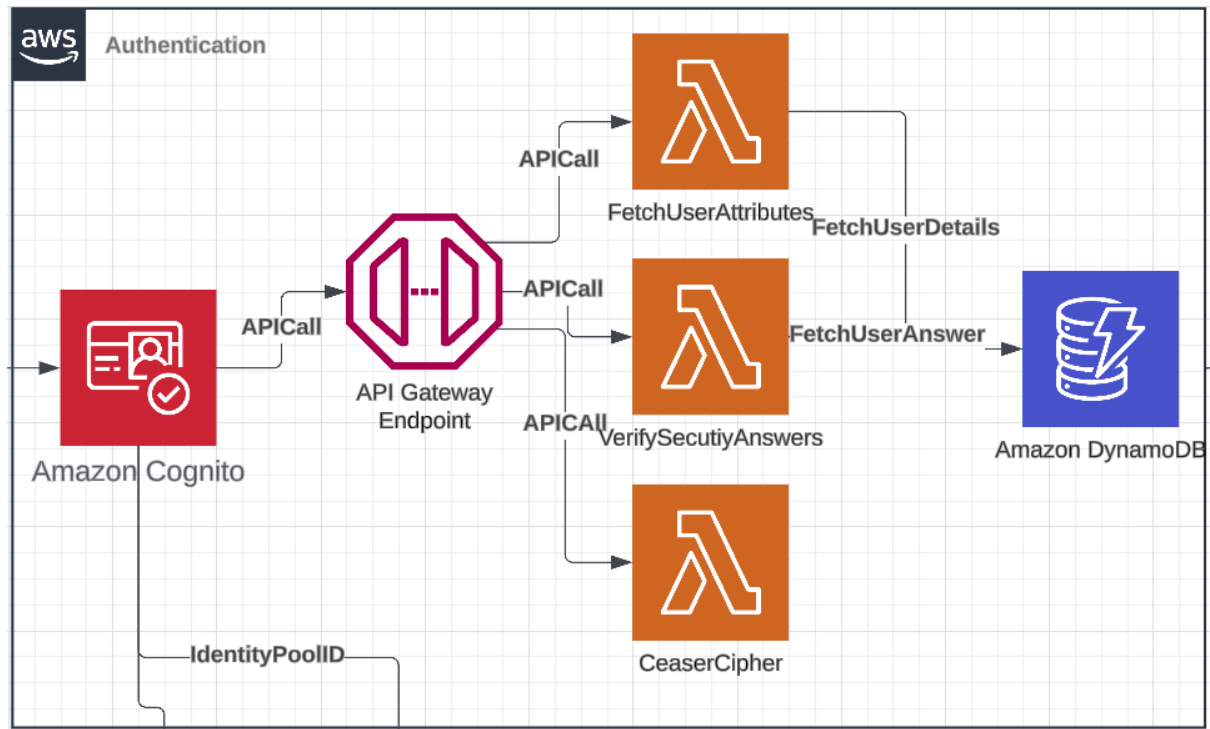


Figure 33: Authentication Architecture Diagram

Test Cases:

1). Integrating the Signin component with the Homepage Navigation bar. So that the user can click on the signIn button

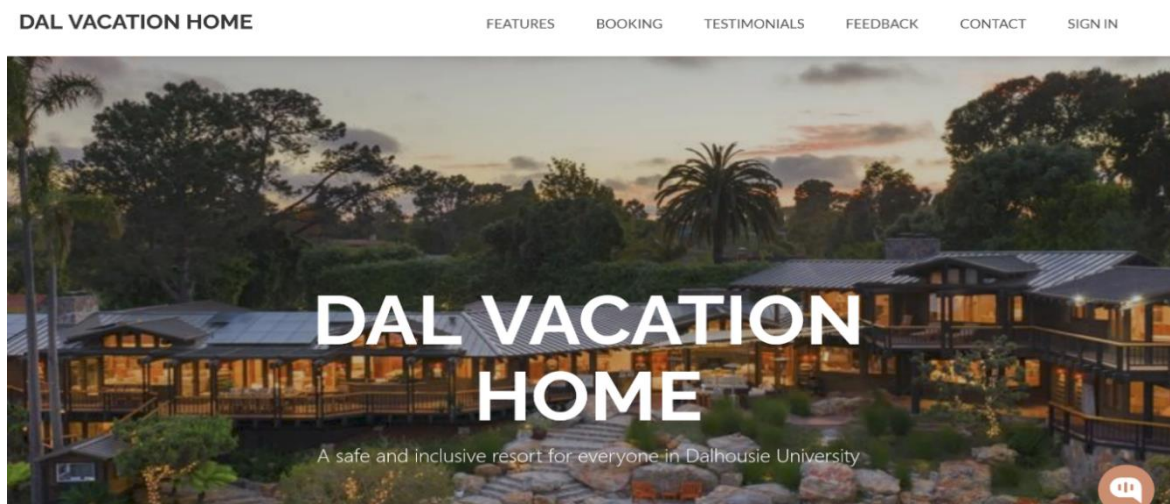
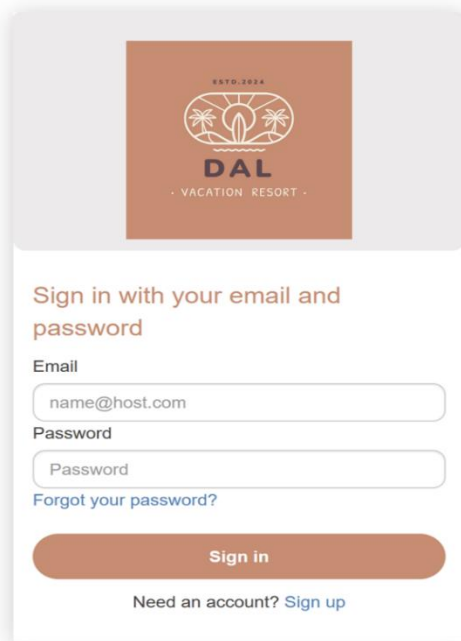


Figure 34: Homepage when user enters the website

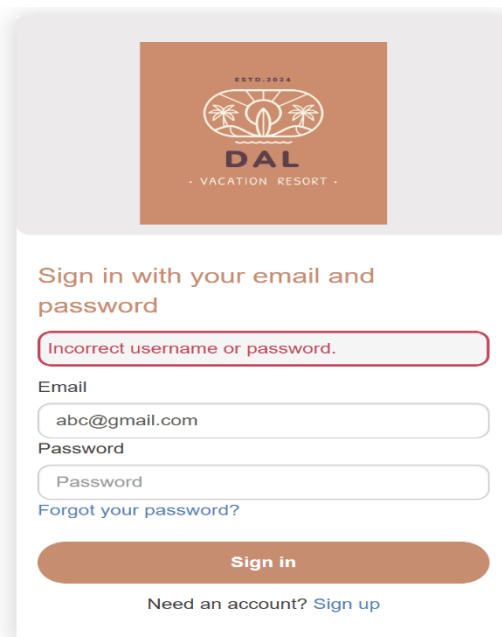
=> Upon Clicking on the signin , the user will be redirected to the signin page, which is shown as below.



The image shows a mobile app sign-in screen for DAL Vacation Resort. At the top is a logo with a palm tree and the text "ESTD. 2024", "DAL", and "VACATION RESORT". Below the logo, the text "Sign in with your email and password" is displayed. There are two input fields: "Email" with the placeholder "name@host.com" and "Password" with the placeholder "Password". A link "Forgot your password?" is located below the password field. A large orange "Sign in" button is at the bottom, with a link "Need an account? Sign up" below it.

Figure 35: Application Sign in Page

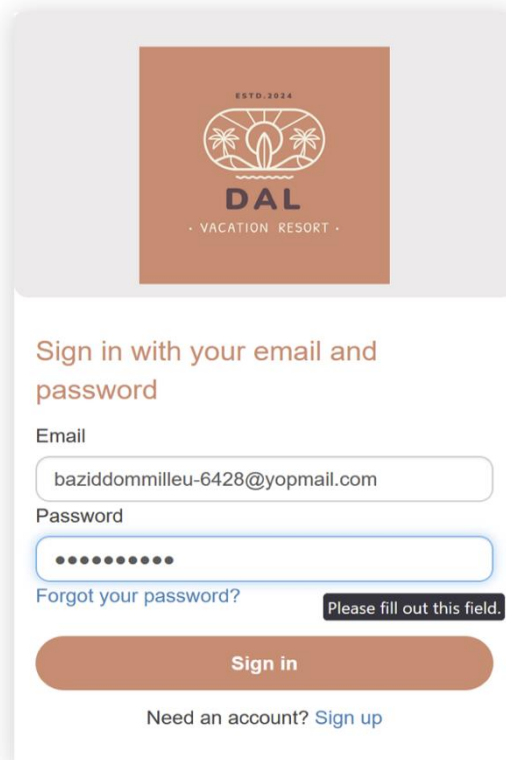
=> If the user credentials are wrong/ User does not exist, then prompts for wrong/invalid credentials.



The image shows the same mobile app sign-in screen as Figure 35, but with an error message. A red-bordered box at the top of the form area contains the text "Incorrect username or password." The "Email" field now contains "abc@gmail.com". The "Password" field still contains "Password". The "Forgot your password?" link and the "Sign in" button are still present, along with the "Need an account? Sign up" link.

Figure 36:User enters incorrect credentials

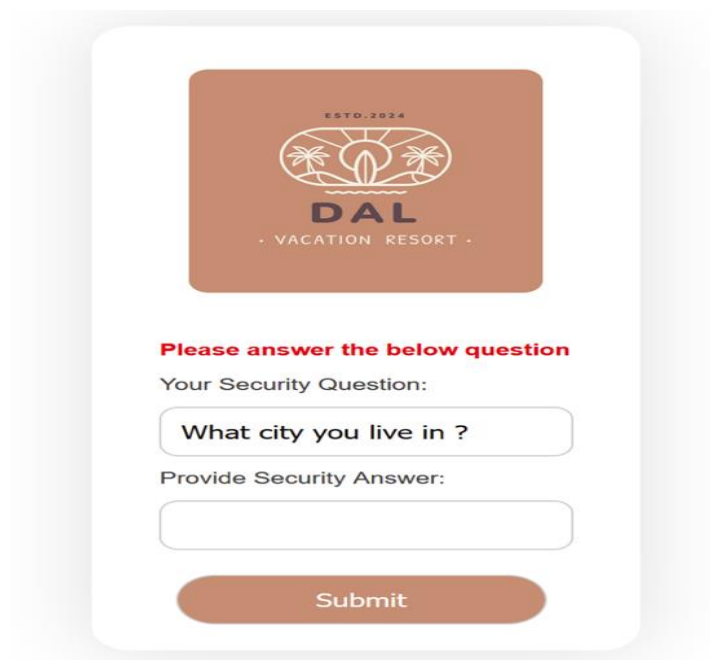
=> User enters the registered credentials to signing to the application:



The login screen for DAL Vacation Resort features a header with the company logo (a palm tree and sun icon) and the text "ESTD. 2024", "DAL", and "· VACATION RESORT ·". Below the header, the text "Sign in with your email and password" is displayed. The form includes an "Email" field with the value "baziddommilleu-6428@yopmail.com" and a "Password" field with masked characters. A "Forgot your password?" link is present, and a black tooltip with the text "Please fill out this field." is visible next to the password field. A large orange "Sign in" button is at the bottom, with a link "Need an account? Sign up" below it.

Figure 37: Entering correct credentials

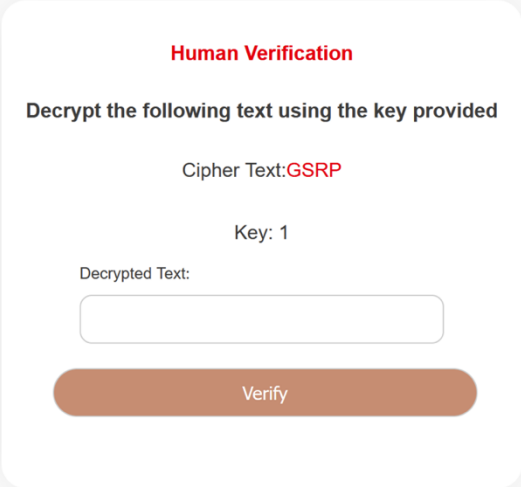
=> Upon successful 1st layer authentication using AWS cognito, the user is redirected to the security answer page where they need to answer the security question they setup during registration :



The security answer page for DAL Vacation Resort features the same header as the login screen. Below the header, the text "Please answer the below question" is displayed in red. The form includes a "Your Security Question:" field with the value "What city you live in ?" and a "Provide Security Answer:" field. A large orange "Submit" button is at the bottom.

Figure 38: Security Answer Page

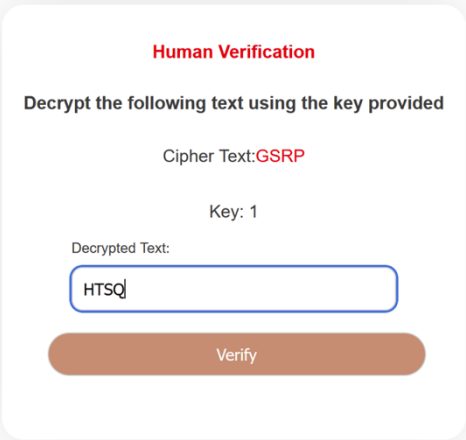
=> Answering the security question correctly, redirects the user to the 3rd authentication layer that is, ceaser cipher verification.



The image shows a mobile app screen titled "Human Verification" in red. Below the title, it says "Decrypt the following text using the key provided". The cipher text "GSRP" is displayed in red. The key "1" is shown below it. A label "Decrypted Text:" is followed by an empty text input field. At the bottom is a large orange "Verify" button.

Figure 39: Decrypt Cipher text

=> User decrypts the cipher using the key provided on the screen.



This image shows the same "Human Verification" screen as Figure 39, but the text input field now contains the decrypted text "HTSQ". The "Verify" button remains at the bottom.

Figure 40 Entering the correct text

=> After successful authentication, the user is redirected back to the homepage where, now the signing button changes to the signout button as the user is fully authenticated.

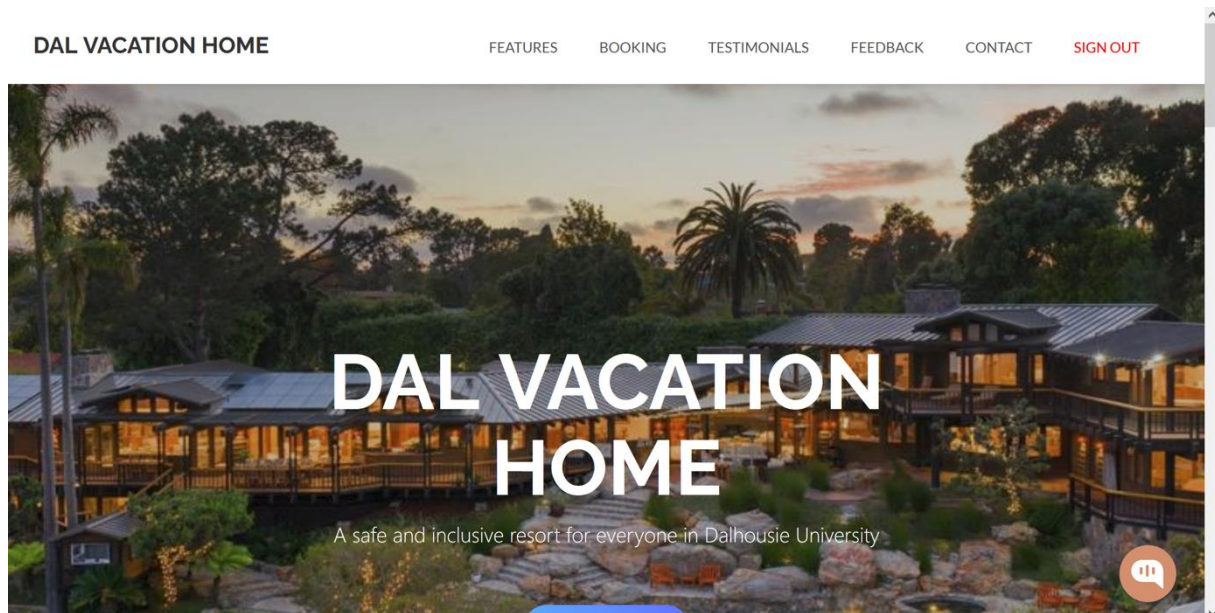


Figure 41 Homepage with signout component

=> If the user wants to sign up for the first time , then he needs to click on the signup button where he will need to enter required details as shown below:

Figure 42 Cognito SignUp page with valid credentials

=> After fulfilling the details, if the user clicks on the signup button , he will be prompted to verify the account :

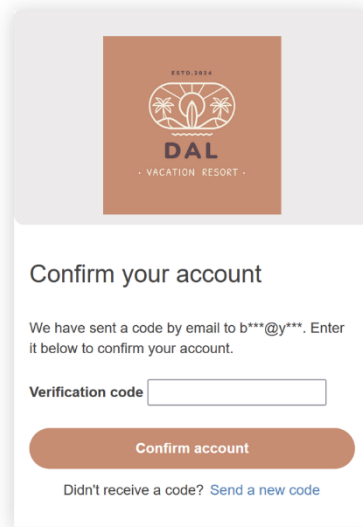
A mobile app interface for account verification. At the top is a logo for 'DAL VACATION RESORT' with 'ESTD. 2024' above it. Below the logo, the heading 'Confirm your account' is displayed. A message states: 'We have sent a code by email to b***@y***. Enter it below to confirm your account.' There is a text input field labeled 'Verification code'. Below the field is an orange button labeled 'Confirm account'. At the bottom, a link says 'Didn't receive a code? Send a new code'.

Figure 43 User account verification code page

=> Below is the verification code that is sent to the user's email:

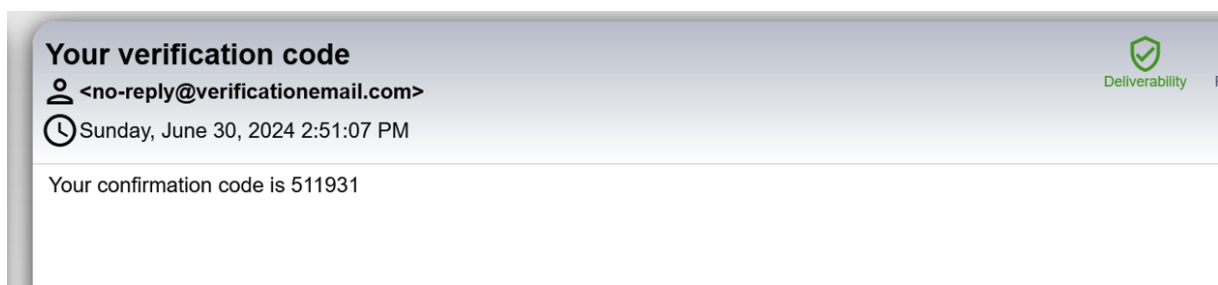


Figure 44 Verification code mail to user

=> Upon entering the verification code, the user will be successfully verified.



Confirm your account

We have sent a code by email to b***@y***. Enter it below to confirm your account.

Verification code

Confirm account

Didn't receive a code? [Send a new code](#)

Figure 45 Enter the correct verification code

=> After successfully adding the verification code, the user will then be prompted to set his security question and answer as shown below and decide the user role based on his requirements:



Security Question:

Security Answer:

Select User Role:

Submit

Figure 46 User security Question Setup page

=> After setting the security factors, the user will then be prompted to signin using his new credentials:

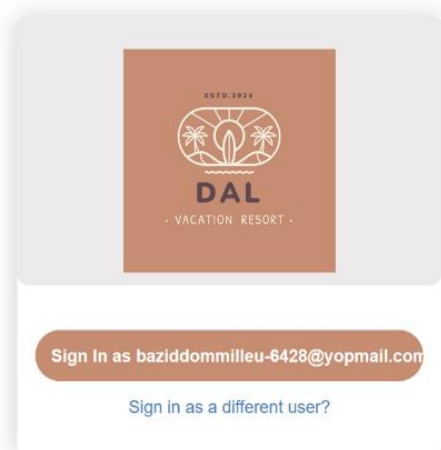


Figure 47 User sign in prompt page

=> For the user security question, if the user enters a wrong answer in the answer input, then the below mentioned error is displayed:

The image shows a user security answer page. At the top, there is a logo for 'DAL VACATION RESORT' with the text 'ESTD. 2024' above it. Below the logo, there is a heading 'Please answer the below question' in red. Underneath, there is a section 'Your Security Question:' followed by a text input field containing 'What city you live in ?'. Below this, there is a section 'Provide Security Answer:' followed by a text input field containing 'Japan'. At the bottom of the form, there is a 'Submit' button. Below the button, there is an error message: 'Error verifying the security answer.'

Figure 48 User security answer page

=> if the user does not satisfy the password requirement, then he will not be able to click the signup button until the password requirement is met.



ESTD. 2024

DAL

· VACATION RESORT ·

Sign up with a new account

Email

Given name

Password

- ✓ Password must contain a lower case letter
- ✓ Password must contain an upper case letter
- ✗ Password must contain a number
- ✓ Password must contain at least 8 characters
- ✗ Password must contain a special character or a space
- ✓ Password must not contain a leading or trailing space

[Sign up](#)

Already have an account? [Sign in](#)

Figure 49 User not satisfying the password requirement

=> What if the user enters a wrong verification code during the registration process



ESTD. 2024

DAL

· VACATION RESORT ·

Confirm your account

Invalid verification code provided, please try again.

We have sent a code by email to t***@c***. Enter it below to confirm your account.

Verification code

[Confirm account](#)

Didn't receive a code? [Send a new code](#)

Figure 50 User enters wrong verification code

=> What if the user enters an incorrect word formed using the key provided, then there will be an alert saying that the incorrect Decryption!

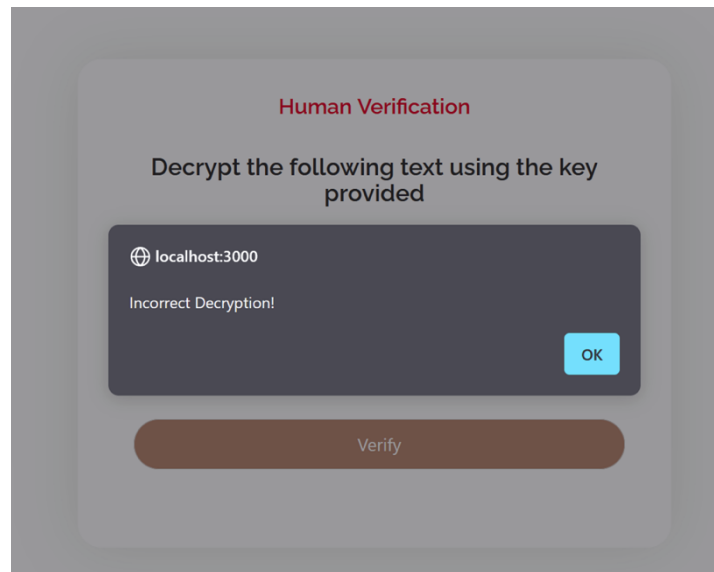


Figure 51 User enters incorrect decrypt code

=> If a user tries to signin with an already registered email address, then he will get error saying user already exists:

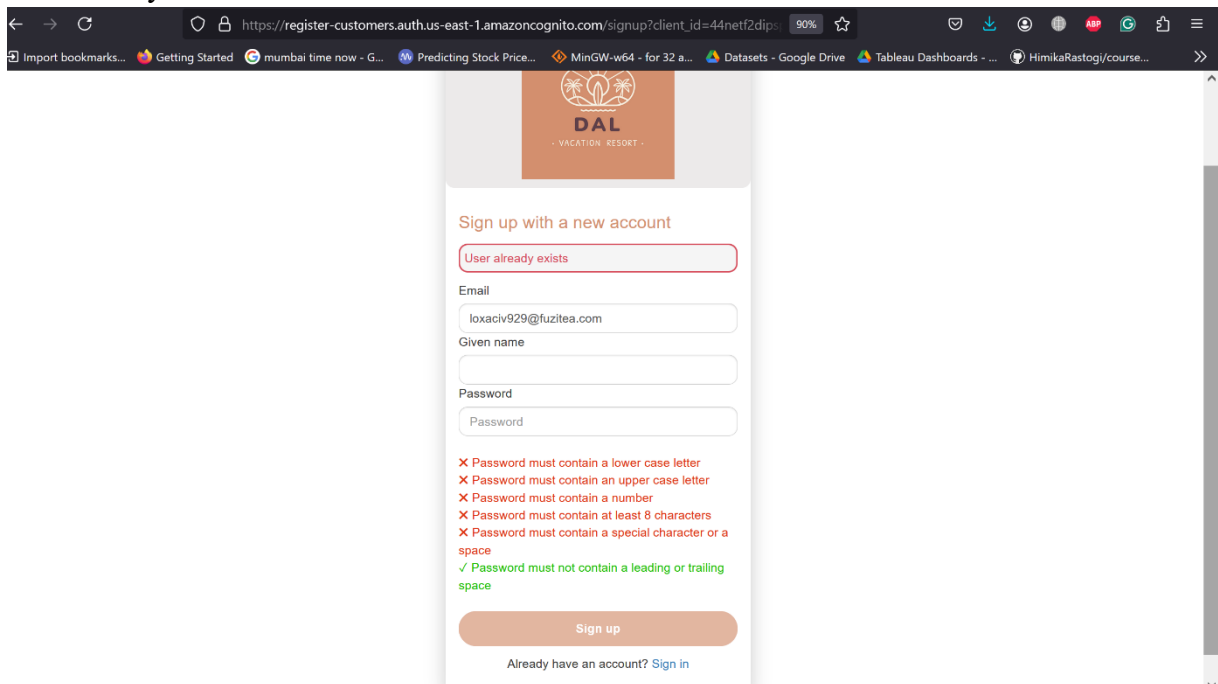


Figure 52 User signup with already registered account

=> Admin side sidebar navigation for the admin to navigate to the different components of the admin dashboard.

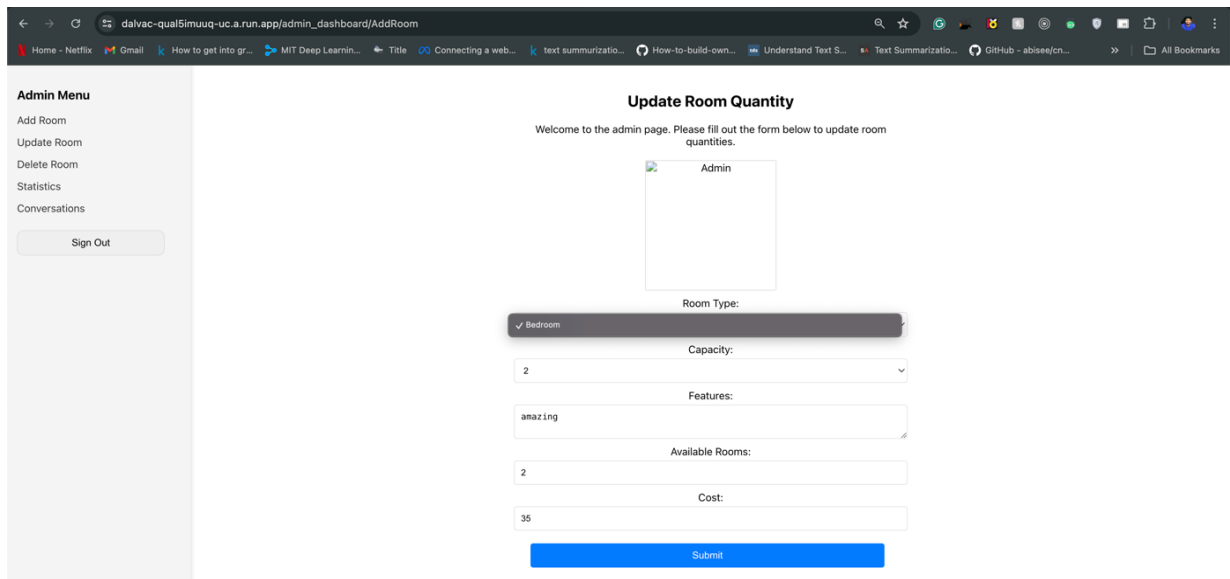


Figure 53 Admin dashboard sidebar navigation

=> Also, the routes are protected from unauthorized access. So, if the users are authenticated, then only one will be able to navigate to different pages else they will be prompted to sign in and authenticate.

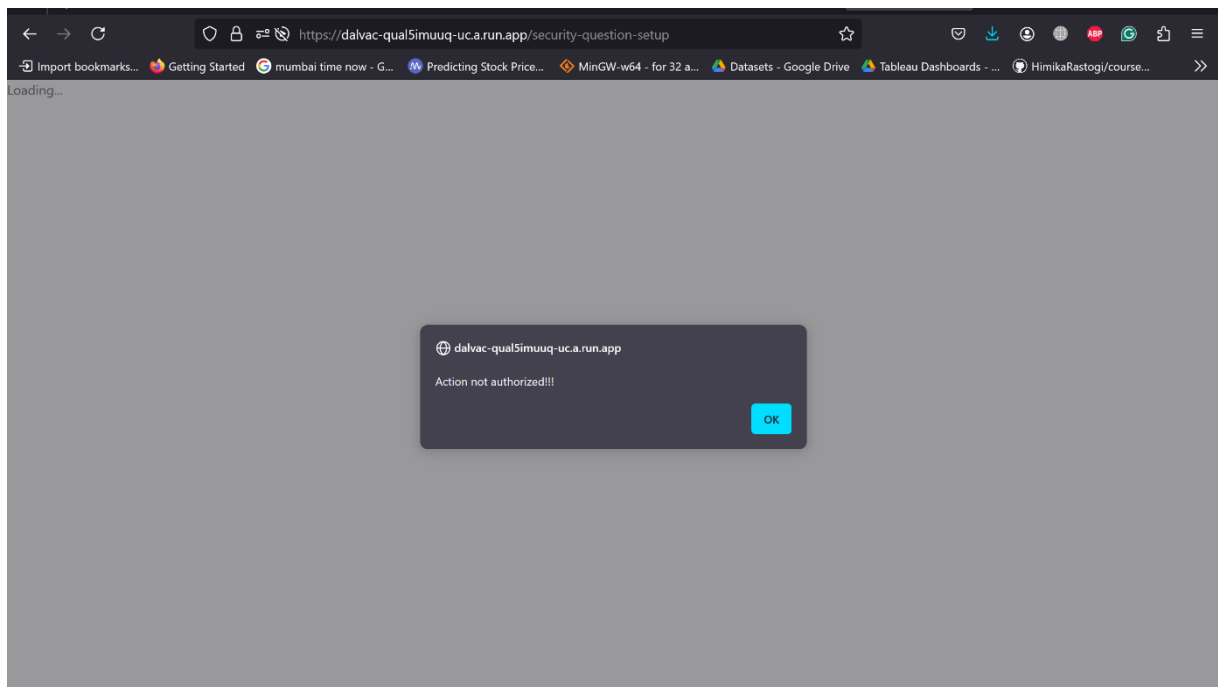


Figure 54 User tries to navigate manually without signin

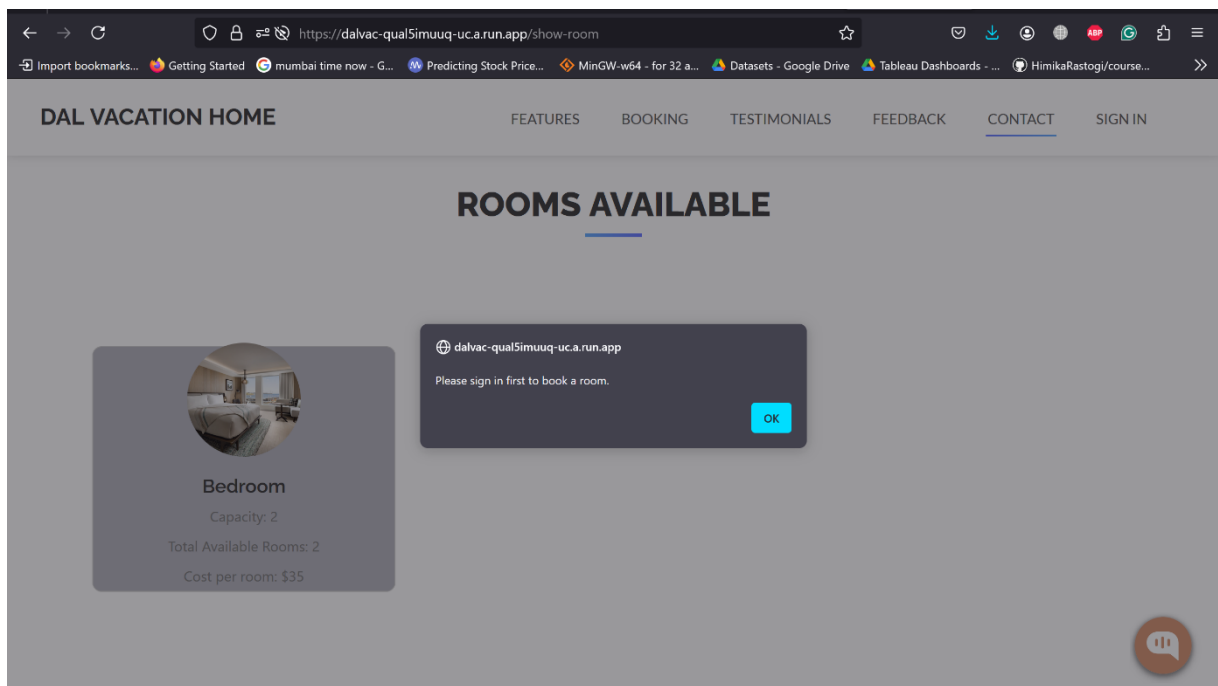


Figure 55 User signup with already registered account

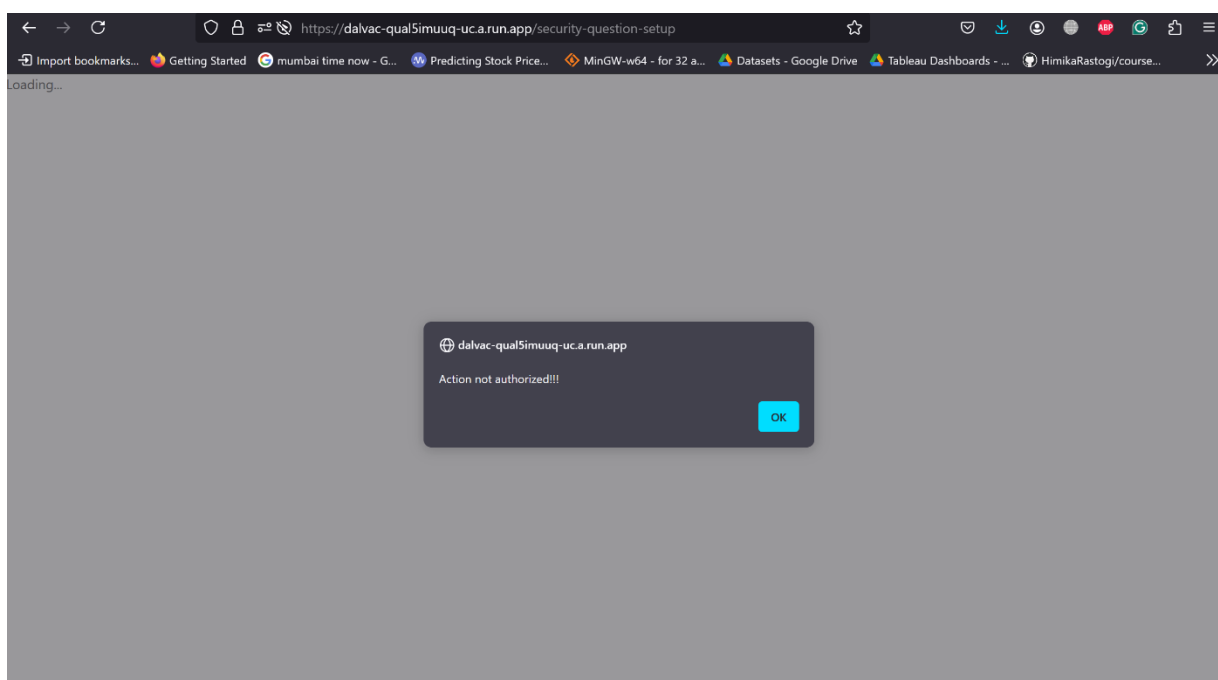


Figure 56 User tries to navigate manually without signin

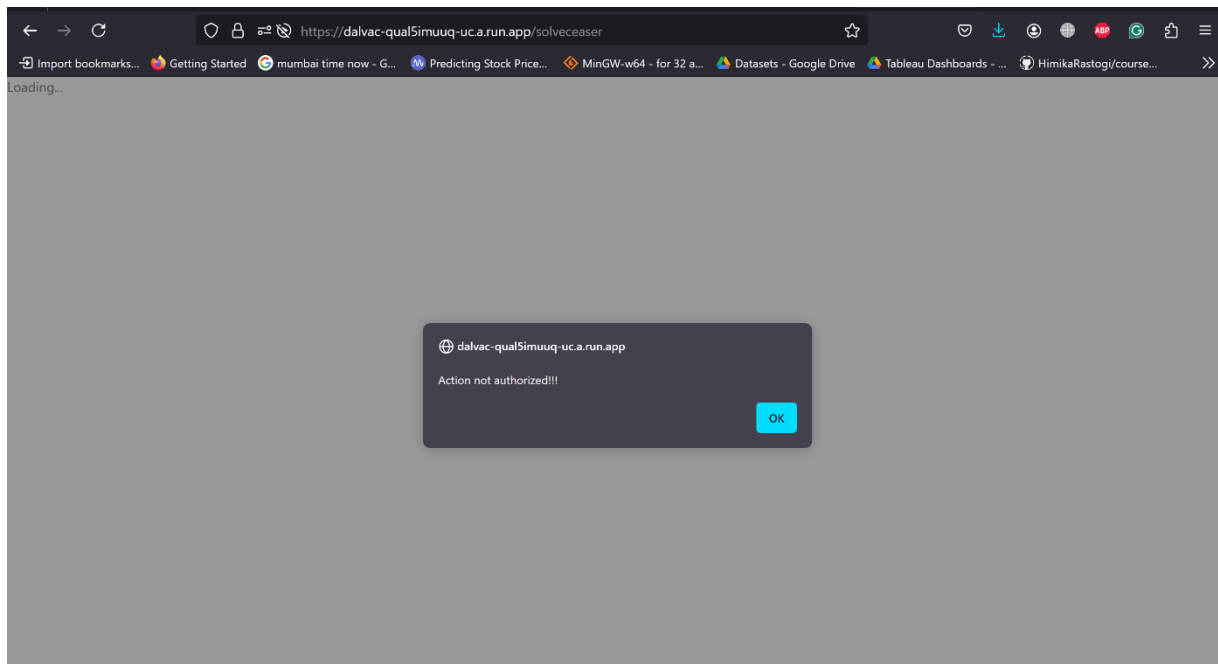


Figure 57 User tries to navigate manually without signin

=> Now coming to the database table side, below are the database tables that we have created in AWS dynamoDB tables.

=> This table stores the users information which they provide for the authentication, such as name , email, security question and answer, etc.

☐	userid (String)	email	email_verified	given_name	hasSecurityQuestion	securityAnswer	securityQuestion	sub	TopicArn	userRole
☐	14c83468-2051-70...	mufakannitteu-8232@yopmail.c...	true	asdfg	true	Halifax	What is your birth pl...	14c83468-...		RegisteredUser
☐	d46884e8-e071-7...	sapsehika@gufum.com	true	sapsehika	true	Halifax	City?	d46884e8-...	arn:aws:sns:...	RegisteredUser
☐	94b8a418-40f1-70...	nicouppiffeddu-2824@yopmail.c...	true	Qwertyu	true	arijit	favourite singer	94b8a418-...		
☐	04583498-4091-7...	seuttautussoge-1872@yopmail...	true	asdfgh	true	Halifax	What is your favouri...	04583498-...		PropertyAgent
☐	b4980478-3041-7...	loxaciv929@fuzitea.com	true	batman	true	Halifax	Which city you live in ?	b4980478-...	arn:aws:sns:...	RegisteredUser

Figure 58 User entries when the user creates and account using MFA

=> Also, we are storing the user logins such as the identity of user as well as the timestamp when they logged in .

Items returned (36)				Actions ▼	Create i
			< 1 > 		
☐	user_id (String) ▼	loginTimestamp (String)			
☐	d46884e8-e071-70bc-387d-d3d47594850e	2024-07-26T17:56:04.423316			
☐	b4980478-3041-70b7-a240-1c4566e7daa3	2024-07-26T13:48:44.445805			
☐	b4980478-3041-70b7-a240-1c4566e7daa3	2024-07-26T13:59:12.905543			
☐	84788428-f091-7088-b8ef-9fb4e391f596	2024-07-25T22:01:38.096110			

Figure 59 User login timestamp gets recorded every time they login to the application

General Tasks [Jaishankar Mohanraj]:

Tasks performed under general tasks for sprint 3:

- Frontend template and setup.

Frontend template and setup:

We used a frontend template that is available in [3]. The setup process included changing the sections contents to the needed contents and images. This also included creating routes to the different pages used for this project.

Technology used:

1. React: It is a frontend framework based on JavaScript, used for development of the frontend for the DalVacationHome.

Create Bookings for rooms:

Objectives:

Registered users need to create bookings for rooms that they want to rent out by creating a BaaS (Backend as a Service). The users should also be able to view the available types of rooms and their features.

Market Research:

A booking system generally uses an SQL database like mySQL but since the objective was to use serverless applications. We used AWS dynamoDB and lambda functions to access the tables. Hence, I have done my research on how to achieve this in a serverless architecture.

To ensure the users know all the different types of rooms and variants of each room, I have thoroughly gone through different hotel booking websites such as Hotel Halifax [4].

Pseudocode:

Getrooms:

- Scan the DynamoDB and get all the rooms.

Bookrooms:

- Scan the available rooms to check if the selected room exists.
- Scan the users table to confirm the user exists.
- Count the number of bookings present on the selected date for the selected room specifications.
- Check if the number of bookings is greater than available rooms, if yes then return error.
- If the number of rooms are lower than available, then create a booking document and return the booking ID.

Technology Stack:

1. AWS DynamoDB: This is used to store all data required for this task. This includes Room details, User details, bookings.
2. AWS lambda: The lambda functions perform backend tasks in the business logic layer such as fetching details from the database, creating new entries, and storing them in the database.
3. API gateway: API gateway is used to group related tasks together and provide a general URL for these tasks.
4. React: The javascript based framework is used to create the frontend for this task.

SQS [Jaishankar Mohanraj]:

Objective: Set up booking queues to ensure race conditions are handled properly and the bookings are created.

Setup When the user requests for a booking a lambda function verifies the user details and the room details from the database, if this information checks out then the lambda function publishes a message to the booking verification queue which triggers a lambda function to check if the room is available on the date that the user has requested, if the room is available then a message is sent to the booking confirmation queue which updates the booking table and calls the send notification lambda function with the details.

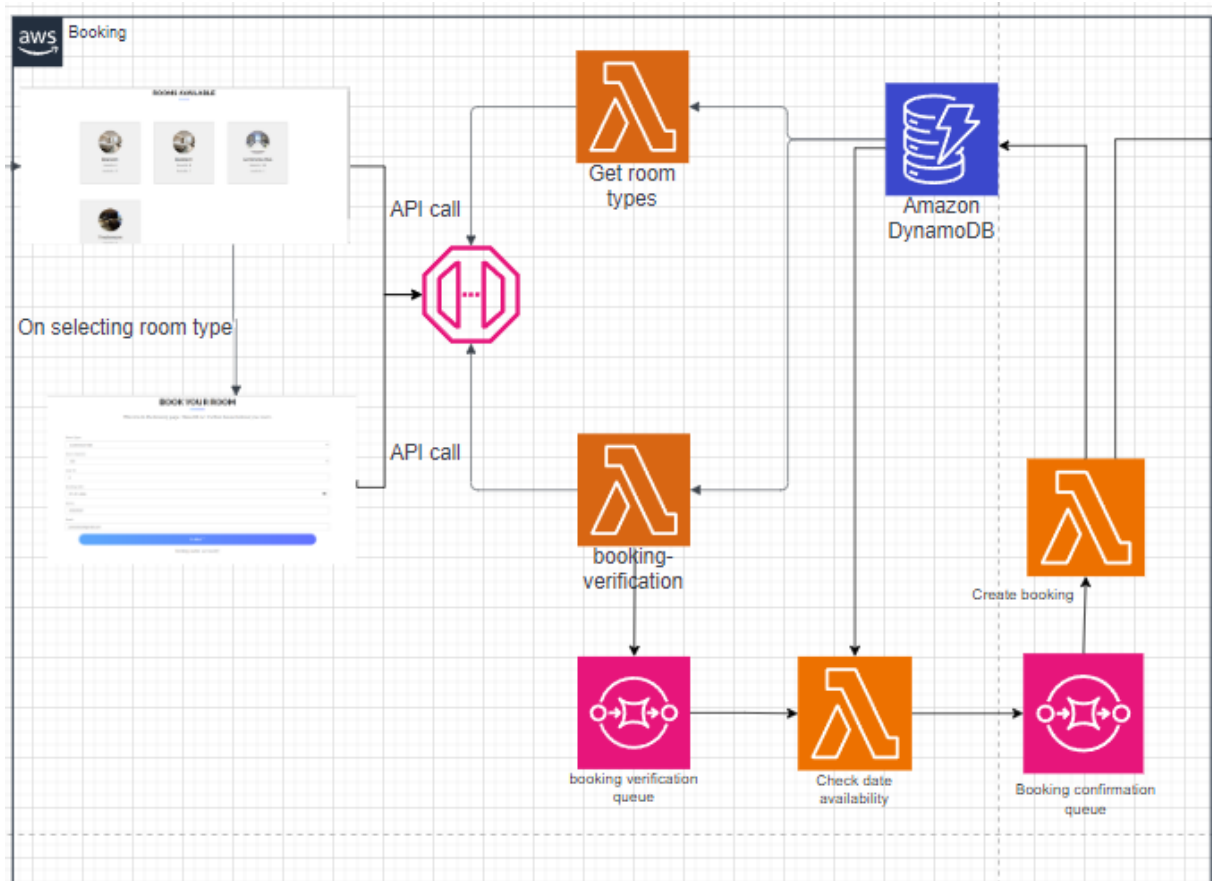


Figure 60: backend architecture for bookings

Notifications [Jaishankar Mohanraj]:

Objective: Set up AWS SNS to send notifications to users for various tasks performed by the user.

Notification instances:

1. On creating a new user, a subscription confirmation email is sent to the user.
2. When the user logs into the website an email is sent to the user along with the timestamp.
3. When the user initiates a booking, an email is sent along with the booking reference number to the user if the booking is successful else, an email is sent stating the reason for booking failure.
4. When the user requests to chat with an agent an email is sent with the link that the user can use to chat with an agent.

Setup: When the user table is updated with a new user, a lambda function is triggered that creates a new SNS topic with the userID and Updates the Record with the generated topic ARN. This

topic ARN is then used to send notifications through an endpoint that triggers a lambda. This setup allows the code for sending notifications to be reused.

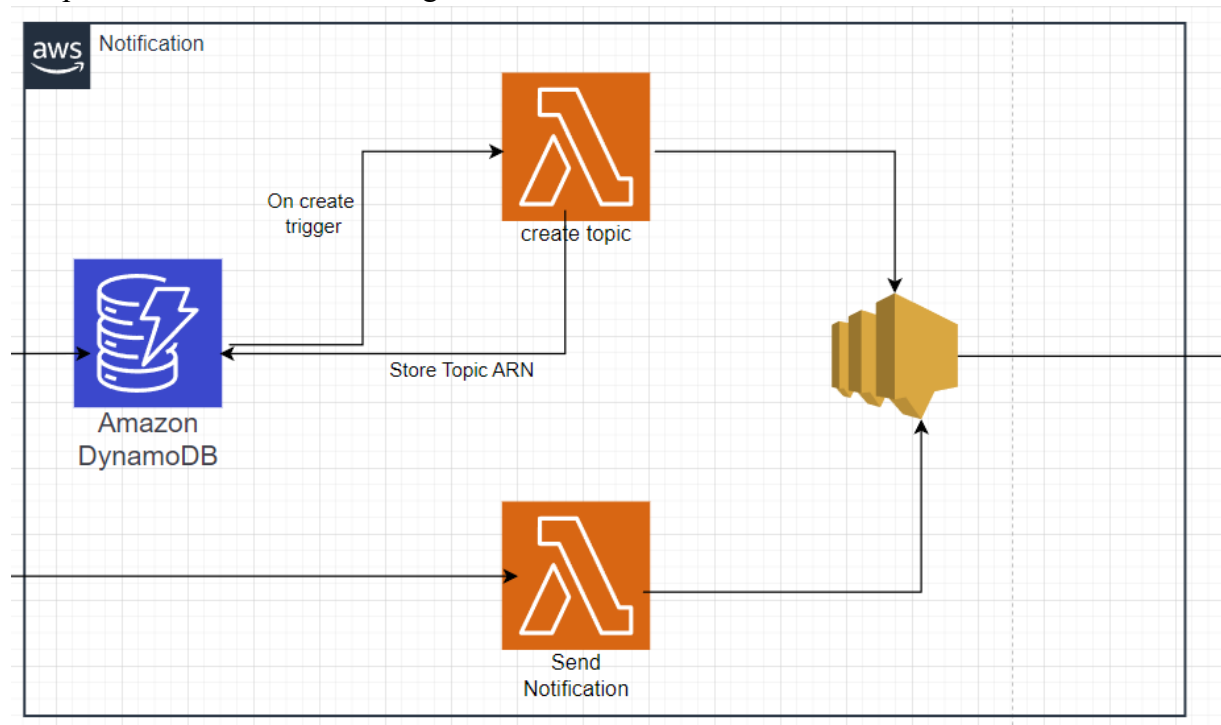


Figure 61: SNS Architecture

Message Passing Module [Jaishankar Mohanraj]:

Objective: A user can submit queries regarding their booking and will be able to get into a live chat with an agent that can address the users concerns.

Market Research: I conducted research in ways to pass information between multiple cloud providers to implement a multicloud approach, how to implement synchronous chat on a website between 2 entities.

To create a chat system effectively and accurately I investigated popular messaging services such as WhatsApp and Facebook messenger. The research aimed to understand the issues with real time chat and how it has been mitigated.

Technology Stack:

1. AWS Lambda: This is used to select a random agent from the pool and communicate with GCP.

2. Google cloud function: This is used to receive data from lambda, publish to pub/sub and store data to firestore.
3. Google Pub/Sub: This is used to ensure that the order of requests is maintained.
4. Google Firestore: This is Used to store the conversations between the agent and the user.
5. Google Firebase Messaging SDK: Used to create the live chat between user and agent.
6. React: The Javascript based framework to create the frontend for this task.

Backend architecture:

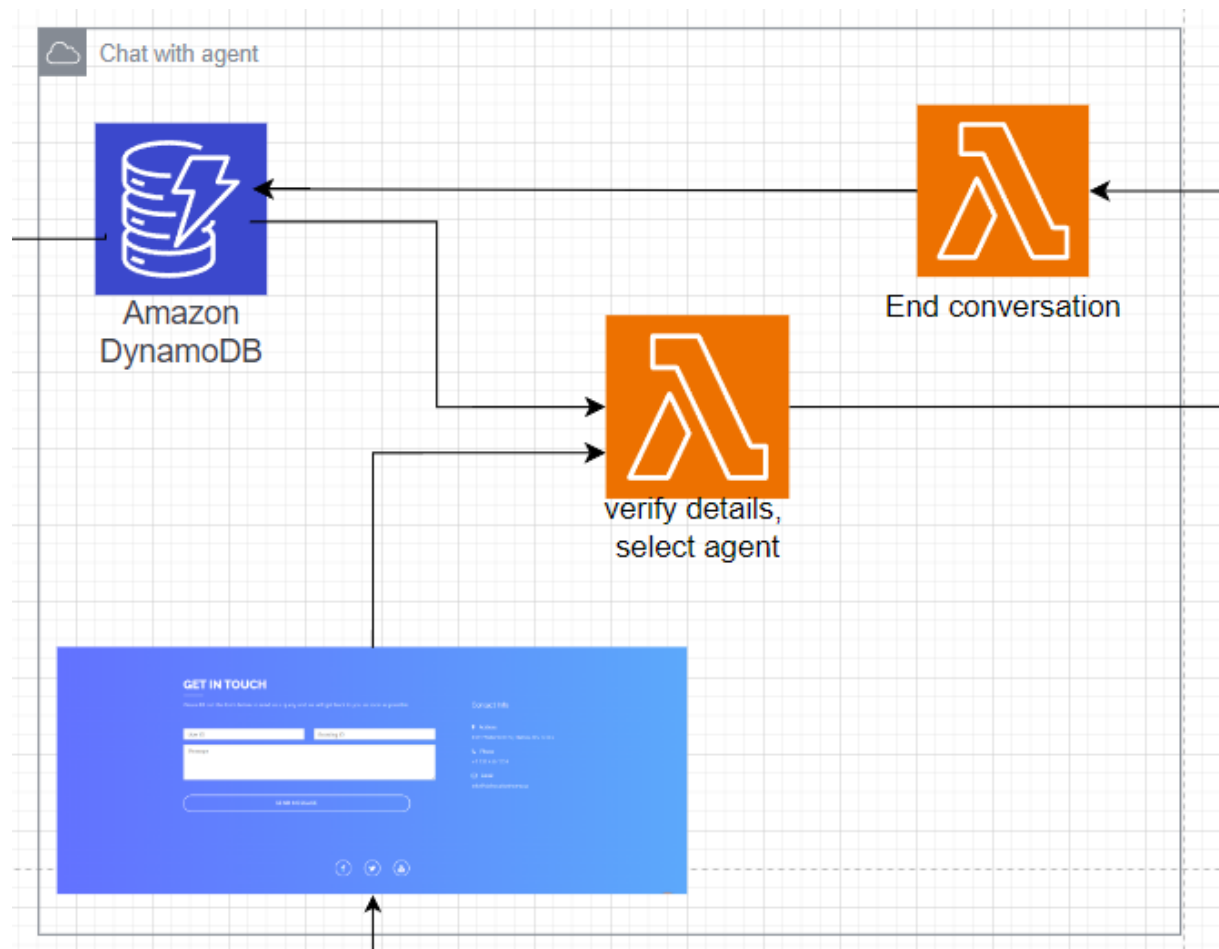


Figure 62: AWS part of the backend for live chat

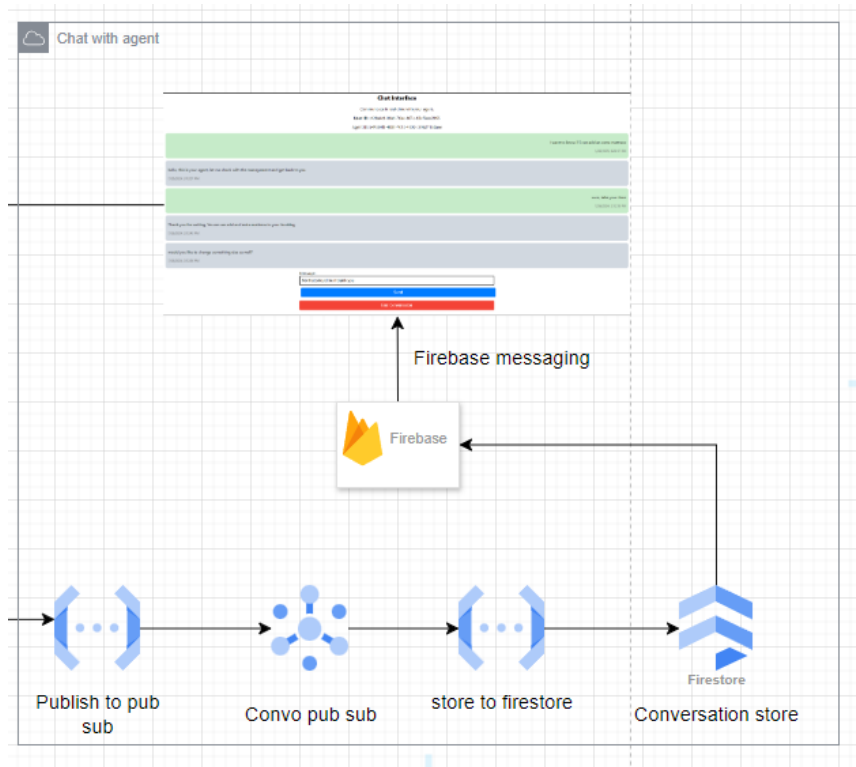


Figure 63: GCP part of the backend for live chat

Individual contribution [Jaishankar Mohanraj]:

My Responsibilities: I was assigned with the task to create a base frontend for the application and perform the base tasks of the application such as viewing rooms and creating bookings. I also worked on notifications and chat with agent.

Home-Page:



Figure 64: Landing page of the website

Features of the resort and bookings:

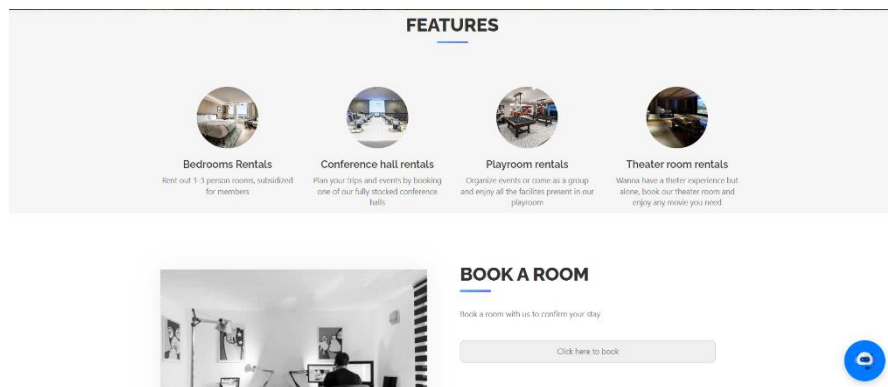


Figure 65: Features of the resort

Room details page:

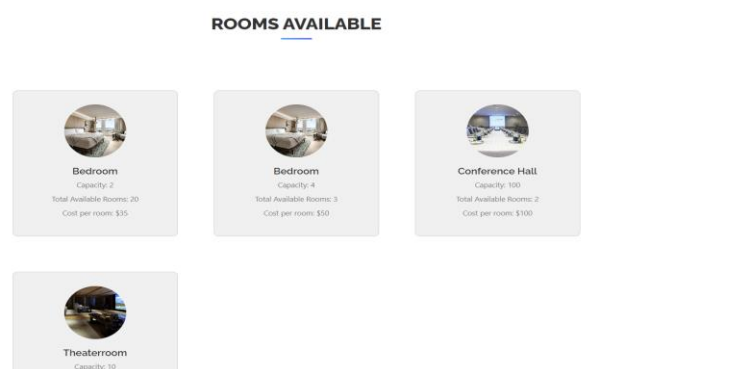


Figure 66: view rooms page

Bookings page and test cases:

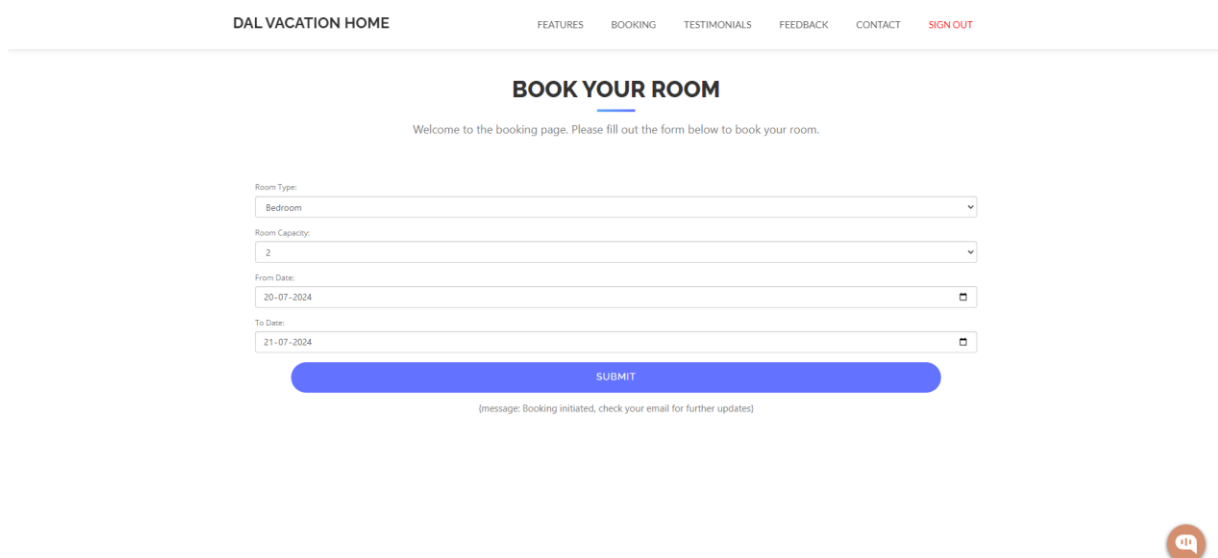


Figure 67: Booking page.

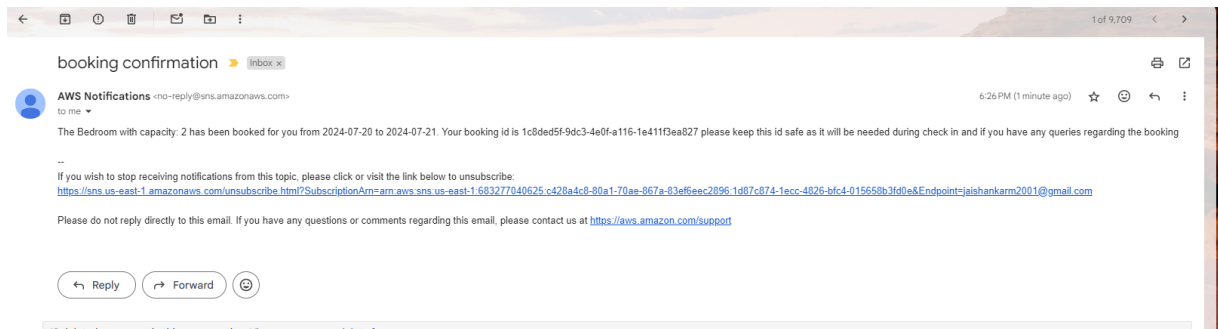


Figure 68: booking confirmation email.

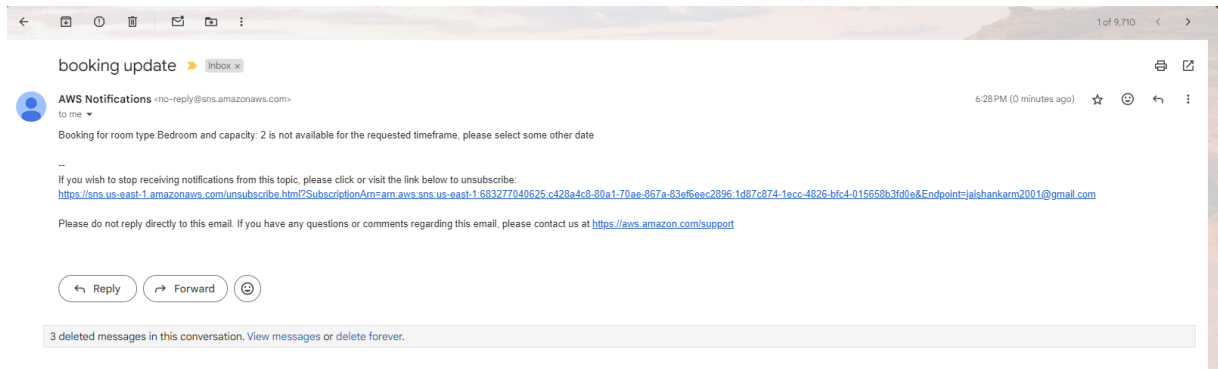


Figure 69: Booking unavailable email.

Message passing test cases:



Figure 70: Agent side active conversation list

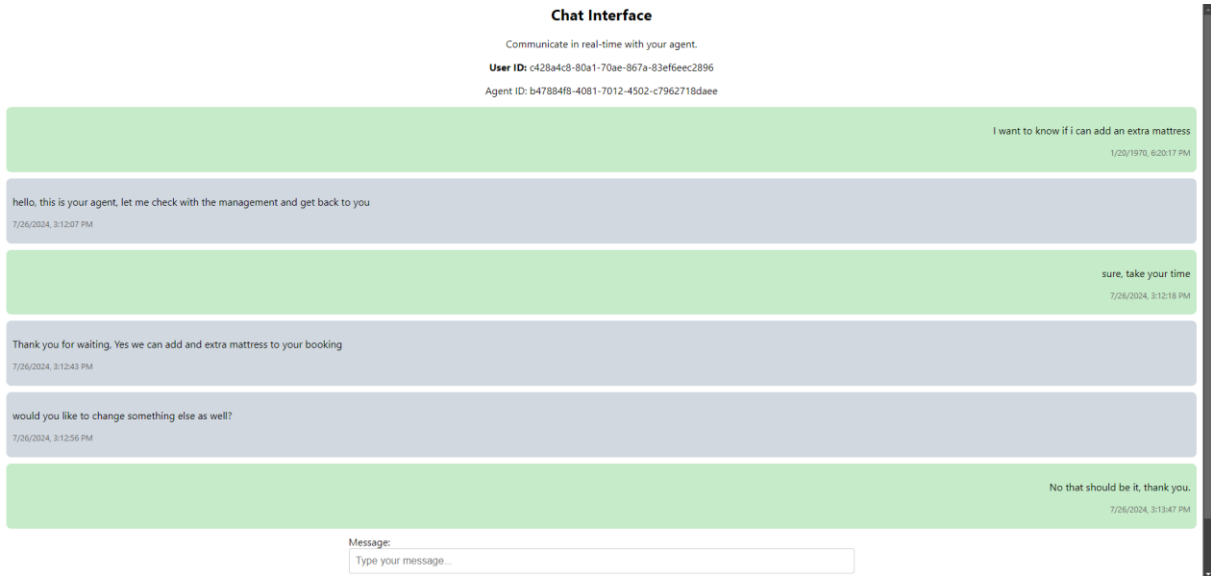


Figure 71: Chat Page

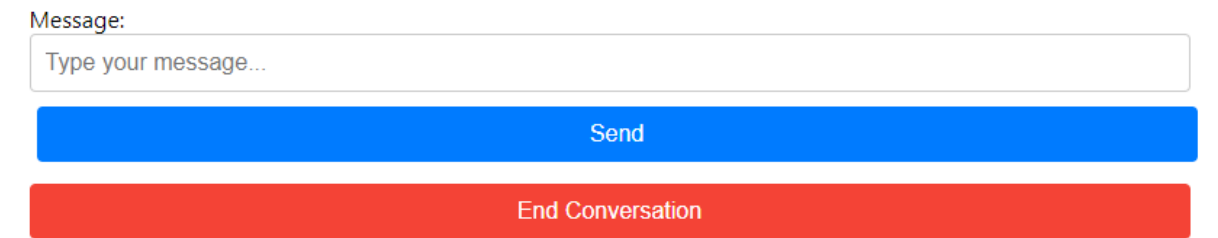


Figure 72: End conversation button

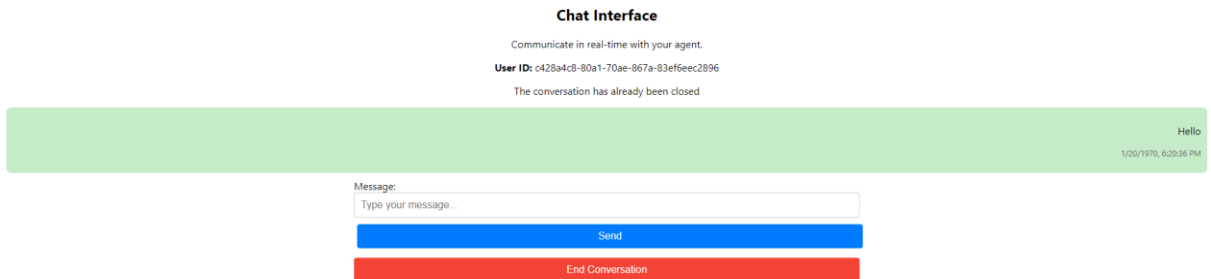


Figure 73: Conversation has already been closed message

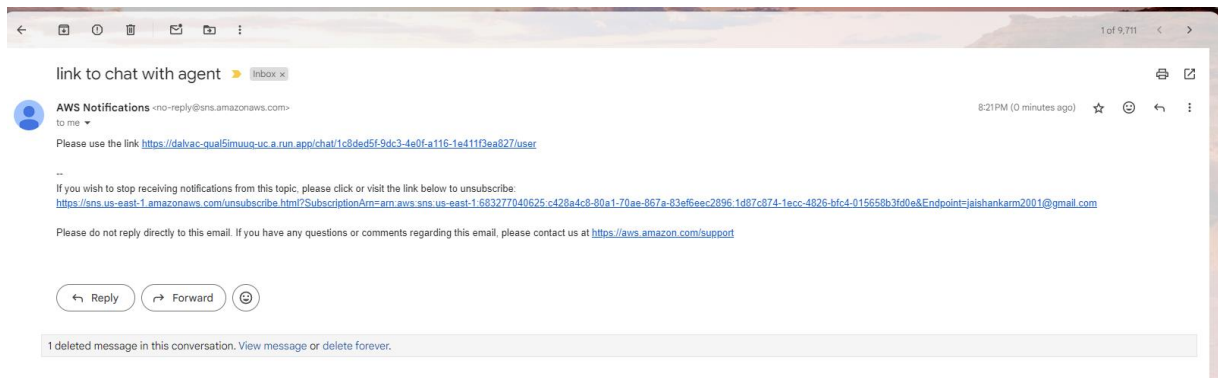


Figure 74: Email sent to user with link for live chat with agent.

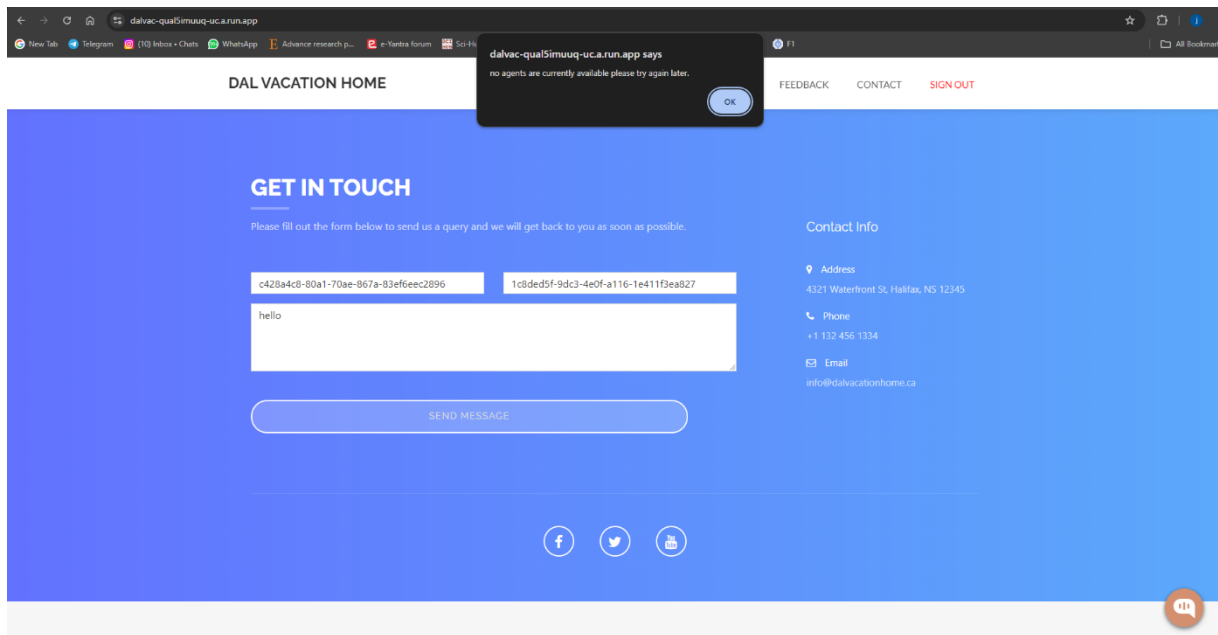


Figure 75: No available agents now message

Login notification:

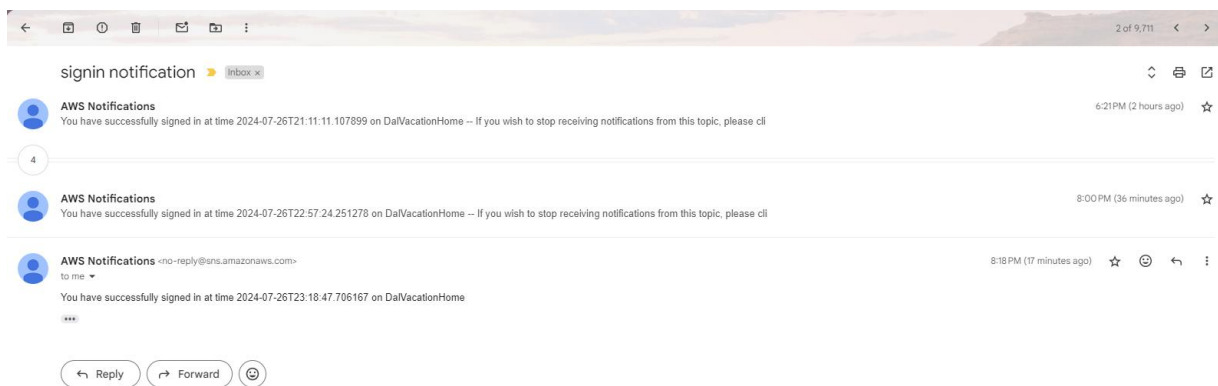


Figure 76: login notifications

GitLab issue board:

As shown below the 3 issues for this task have been completed and merged to main.

Booking

Edit ⋮

Open Issue created 1 day ago by Jaishankar Mohanraj

Implement booking of room for user. Ensure to use a serverless backend. test if user exists, if room exists and if all the available rooms of that type for that particular date is already booked or not

Edited right now by Jaishankar Mohanraj

👍 0 🙏 0 😊

Create merge request ▾

📁 Drag your designs here or [click to upload](#).

Child items ✓ 3

Show labels ☒ Add ▾ ^

- | | |
|---|---------------|
| ☐ Backend
Complete | 🕒 Sprint 2 X |
| ☐ frontend
Complete | 🕒 Sprint 2 X |
| ☐ integration
Complete | 🕒 Sprint 2 X |

Figure 77: Booking issue.

Live chat with agent

Edit ⋮

Closed Issue created 3 hours ago by Jaishankar Mohanraj

👍 0 🙏 0 😊

🔄 Checking branch availability...

📁 Drag your designs here or [click to upload](#).

Child items ✓ 4

Show labels ☒ Add ▾ ^

- | | |
|---|---------------|
| ☐ Select random agent from pool
Closed | 🕒 Sprint 3 X |
| ☐ push data to GCP and store in firestore
Closed | 🕒 Sprint 3 X |
| ☐ real time chat UI
Closed | 🕒 Sprint 3 X |
| ☐ Chat backend
Closed | 🕒 Sprint 3 X |

Linked items 📁 0

Add ^

Link issues together to show that they're related or that one is blocking others. [Learn more](#).

Figure 78: Chat module

Notifications

Edit ⋮

Closed Issue created 3 hours ago by Jaishankar Mohanraj

👍 0 🙏 0 😊

📁 Drag your designs here or [click to upload](#).

Child items ✓ 3

Show labels ☒ Add ▾ ^

- | | |
|--|---------------|
| ☐ Create topic on signup
Closed | 🕒 Sprint 3 X |
| ☐ notifications endpoint
Closed | 🕒 Sprint 3 X |
| ☐ login notification
Closed | 🕒 Sprint 3 X |

Linked items 📁 0

Add ^

Link issues together to show that they're related or that one is blocking others. [Learn more](#).

Figure 79: Notifications

Data Analysis and Visualization: [MD Samshad Rahman]

I was responsible for implementing a serverless architecture for data analysis and visualization. As a result, I implemented a feedback feature and admin dashboard where admins can check user's login statistics.

Objectives:

- Non-registered or not logged-in users will be able to read all the feedback already submitted, along with their polarity, in a tabular format.
- Logged-in users can submit feedback. Additionally, they can check their submitted feedbacks and delete if they want.
- Logged-in admin can see a dashboard with users login statistics along with the total user count.

Market research:

I conducted research on effective strategies for decoupling system architectures to enhance performance, scalability, and maintainability. Based on my findings, I have chosen to implement the following approaches:

Function-Specific Tasks: Decoupling systems into smaller, single-responsibility functions increases modularity and simplifies debugging. So, I divided the entire system into smaller tasks, ensuring each function is responsible for only one task. This approach allows for better scalability and easier maintenance.

Handler Functions for Task Management: Utilizing handler functions to manage tasks allows for more organized and efficient processing of requests. Handler functions were employed to streamline the task management process, ensuring tasks are executed efficiently and effectively.

Dashboard Update with Lambda and EventBridge: Regularly updating dashboard data is crucial for real-time monitoring and reporting. Using scheduled tasks ensures data is consistently refreshed without manual intervention. A Lambda function with a scheduler using EventBridge was written to update the dashboard data every 5 minutes (or as specified). This approach ensures that the dashboard remains current and provides accurate insights.

System Architecture for Data analysis (Feedback):

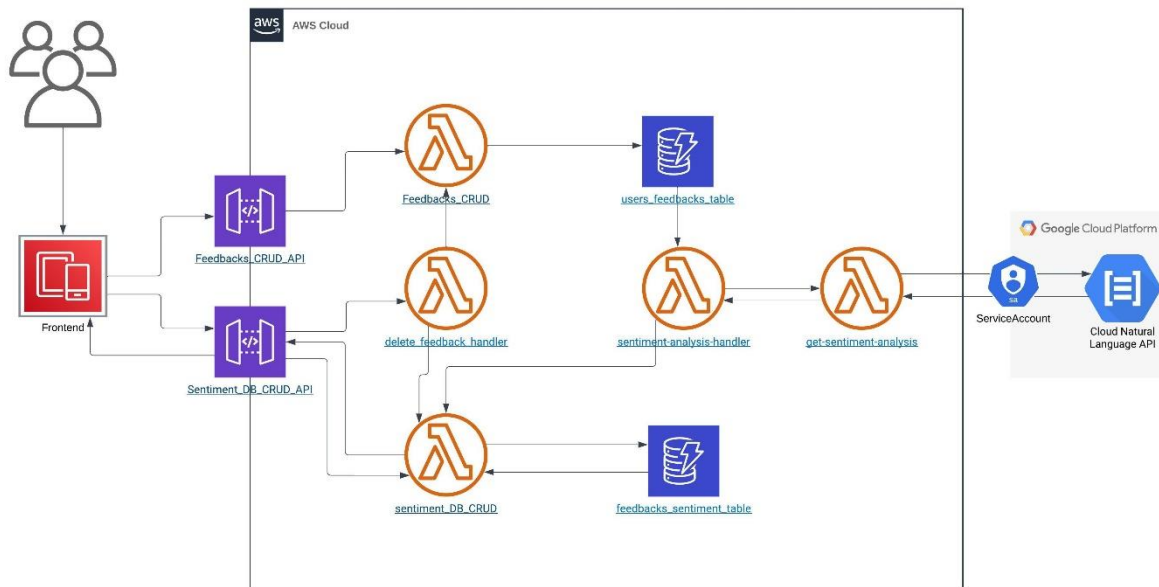


Figure 80: System architecture for feedback

Explanation:

I have worked on a serverless solution for processing feedback submission/retrieval from database and analyze the feedback with sentiment analysis. Writing a brief explanation of the architecture.

1. Frontend Interaction:

- Users submit feedback through the front-end application (React App).
- The frontend makes HTTP requests to the backend APIs.
- Users can fetch/see all the feedbacks with their polarity in a tabular format.

2. Backend APIs:

- Feedbacks_CRUD_API: This API handles inserting, reading, and deleting feedback entries inside users_feedbacks_table (DynamoDB table).
- Sentiment_DB_CRUD_API: This API handles inserting, reading, and deleting feedback entries inside feedbacks_sentiment_table (DynamoDB table).

3. Lambda Functions:

Feedbacks_CRUD:

- When a user submits feedback, this Lambda function is triggered.
- It stores the feedback data into the users_feedbacks_table DynamoDB table.
- This insert will also trigger the sentiment-analysis-handler to analyze the sentiment of the new feedback.

delete_feedback_handler:

- This function is responsible for deleting feedback entries.
- When a delete request is received, it calls DELETE API with the feedback_id to both feedbacks_CRUD and sentiment_DB_CRUD to delete the feedback entries from users_feedbacks_table and feedbacks_sentiment_table.

sentiment-analysis-handler:

- This function is triggered when new feedback is created.
- It sends the feedback text to get-sentiment-analysis (Lambda) for sentiment analysis.
- It stores the analyzed sentiment data into the feedbacks_sentiment_table.

get-sentiment-analysis:

- This function gets the feedback text and sends the feedback text to the Google Cloud Natural Language API for sentiment analysis and returns the polarity and score.

4. DynamoDB Tables:

- users_feedbacks_table: Stores the raw feedback data from users.
- feedbacks_sentiment_table: Stores the sentiment analysis results of the feedback along with feedback data.

Test cases:

Test case 1:

Non-logged-in users can see all the existing feedbacks in a tabular format.

DAL VACATION HOME

FEATURES BOOKING TESTIMONIALS FEEDBACK CONTACT SIGN IN

Home

All Feedbacks

Refresh

Date Time	Feedback	Polarity
7/25/2024, 9:25:55 PM	what is this horrible hotel?	negative
7/25/2024, 9:24:25 PM	very good resort, had fun	positive
7/19/2024, 10:47:50 PM	The staffs and their co ordination communication were poor	negative
7/19/2024, 10:43:43 PM	Felt that the receptionist on the desk should have known about Wienhbf Surely you would know about your own main station and how to get to it	negative
7/19/2024, 10:43:36 PM	room and bathroom door are old room key is not working well because the door is broken TV channels are limited	negative

First Prev 1 Next Last

Log in to give feedback

Figure 81 All existing feedbacks with polarity

Test case 2:

Logged-in users can see their own feedback with polarity and a delete option.

DAL VACATION HOME FEATURES BOOKING TESTIMONIALS FEEDBACK CONTACT SIGN OUT

First Prev 1 Next Last

Submit Feedback

Feedback:

Submit

My Feedbacks

Refresh

Date Time	UserID	Feedback	Polarity	Delete Feedback
7/19/2024, 10:47:50 PM	a4f8b488-a091-7063-e1e6-a775a15fcec7	The staffs and their co ordination communication were poor	negative	Delete
7/19/2024, 10:43:43 PM	a4f8b488-a091-7063-e1e6-a775a15fcec7	Felt that the receptionist on the desk should have known about Wienthof Surely you would know about your own main station and how to get to it	negative	Delete
7/19/2024, 10:43:36 PM	a4f8b488-a091-7063-e1e6-a775a15fcec7	room and bathroom door are old room key is not working well because the door is broken TV channels are limited	negative	Delete
7/19/2024, 10:43:29 PM	a4f8b488-a091-7063-e1e6-a775a15fcec7	It was a quirky modern hotel the themed lifts were fun It was clean and the location was fabulous	positive	Delete
7/19/2024, 10:43:00 PM	a4f8b488-a091-7063-e1e6-a775a15fcec7	This is the worst test text and give very negative, sad and depressing vibe.	negative	Delete

First Previous 1 Next Last



Figure 82 Logged-in user's own feedbacks with delete button

Test case 3:

Logged-in users can submit feedback.

Submit Feedback

Feedback:

very nice staffs the room is big and clean the location is good and the breakfast is tasty

Submit

Feedback submitted successfully!

Figure 83 Submit feedback

Test case 4:

Logged-in users can delete feedback.

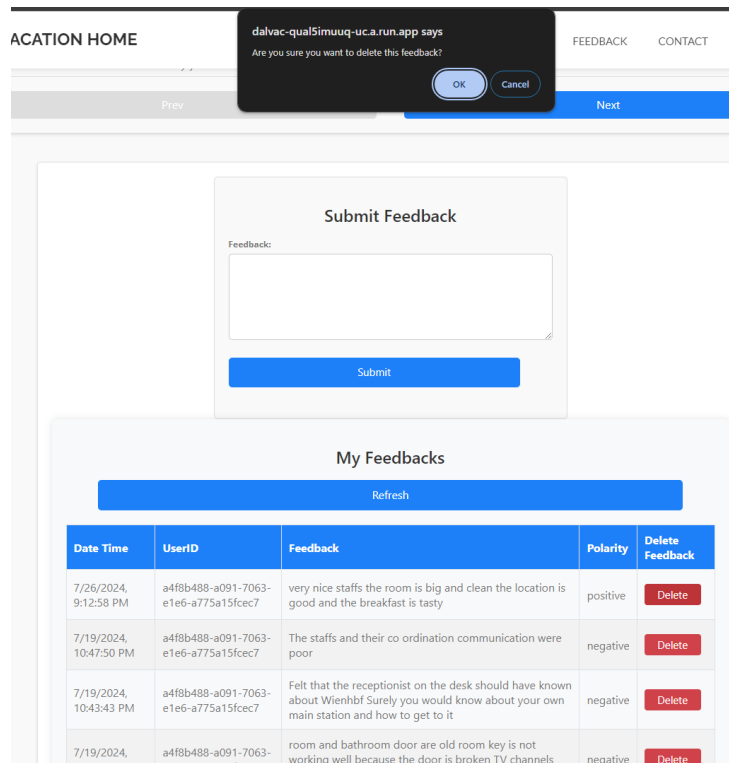


Figure 84 Delete feedback

System Architecture for Data visualization (Admin Dashboard):

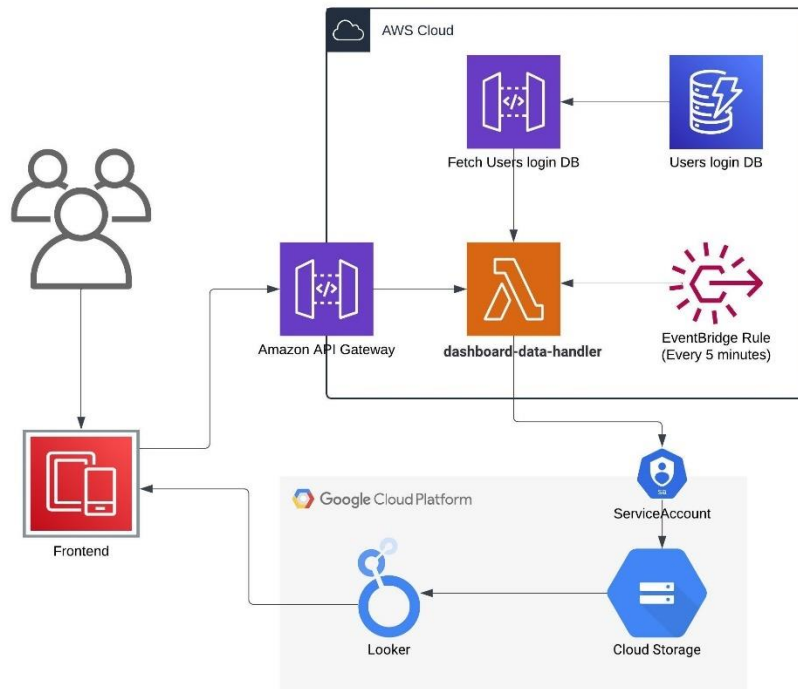


Figure 85 System Architecture for Data visualization (Admin Dashboard)

Explanation:

1. **User Interaction:** Users interact with the React frontend, which renders the dashboard using an iframe.
2. **API Gateway Trigger:** When an admin logs in system sends a GET call to dashboard-data-handler function to trigger and update the dashboard data.
3. **Dashboard Data Handler:** This function handles the dashboard data and has two key connections:
 - **EventBridge Rule:** Triggers every 5 minutes to update data automatically.
 - **Google Cloud Platform:** Dumps the user login data as a csv for data analysis to the Cloud Storage.
4. **Looker Studio:** Reads the csv file from Cloud Storage and renders the dashboard for admins. By default, the Looker studio updates the data every 15 minutes.

Test cases:

Test case 1:

Dashboard renders on the front end for admins.

Total Users		Record Count	
16		53	

user_id	Record Count
1. c428a4c9-03a1-70a6-817a-03a7a4cc2894	13
2. 84684489-80a1-703a-7b6b-9f38a4c8722c	7
3. 840804a9-w3c1-707b-682b-70c7a06b844	6
4. 44a8b489-02d1-70d7-f2b9-930a47f6b0b2	8
5. 44a8b489-02d1-70d9-171a-609b79216468	4
6. 44b82489-45b1-70a7-4cac-167d7f93a3f	3
7. 44f8b489-w091-70a3-a1a9-a778a18fca47	3
8. b4980479-03a1-70a7-a3a0-1a486a70a23	2
9. b478b479-w091-7012-482c-c79a2715a4a4	2

loginTimestamp
1. Jul 26, 2024, 9:29:00 PM
2. Jul 26, 2024, 9:23:00 PM
3. Jul 26, 2024, 9:20:01 PM
4. Jul 26, 2024, 9:19:02 PM
5. Jul 26, 2024, 9:18:18 PM
6. Jul 26, 2024, 9:13:43 PM
7. Jul 26, 2024, 9:11:11 PM
8. Jul 26, 2024, 9:10:18 PM
9. Jul 26, 2024, 8:34:18 PM

Figure 86 Admin Dashboard

Test case 2:

When an admin logs in manually lambda gets triggered to update csv using API Gateway.

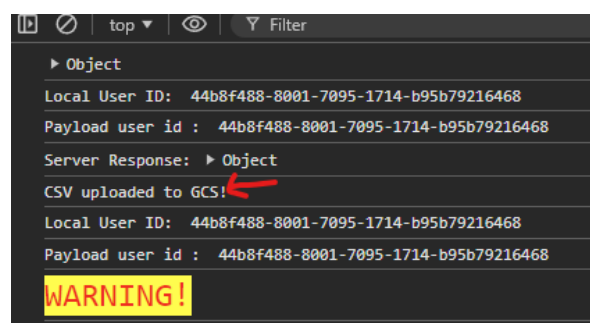


Figure 87 Manual lambda trigger to update CSV

Test case 3:
EventBridge triggers every 5 minutes to update the data.

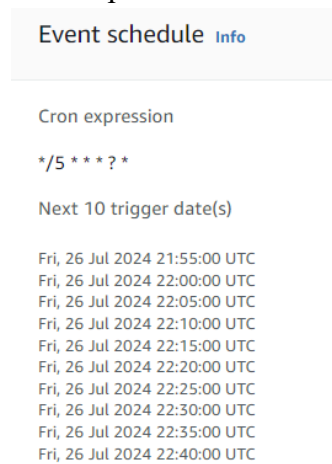


Figure 88 Cron job expression

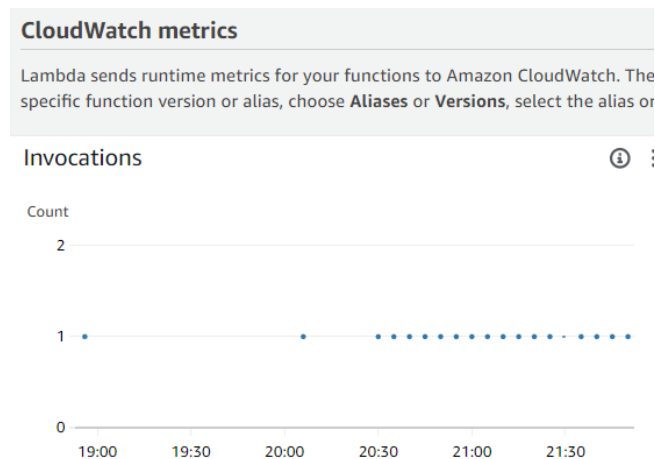


Figure 89 Lambda Invocation every 5 minutes

Deployment:

IaC:

We researched cloudRun to deploy our application on.

We created a terraform IaC script to spin up our cloudRun resource using this script and deployed our containerized application on GCP. The docker image was deployed on dockerhub. This image is pulled in our terraform script.

```
}
}

# google_cloud_run_service iam_member.month will be created
resource "google_cloud_run_service_iam_member" "month" {
  tag = (known after apply)
  id = (known after apply)
  location = "us-central1"
  member = "allUsers"
  project = (known after apply)
  role = "roles/run.invoker"
  service = "newtrial"
}

Plan: 2 to add, 0 to change, 0 to destroy.

Changes to Outputs:
  cloud_run_url = (known after apply)

Do you want to perform these actions in workspace "gcp-demo-sha"?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

google_cloud_run_service.default: Creating...
google_cloud_run_service.default: Still creating... [10s elapsed]
google_cloud_run_service.default: Still creating... [20s elapsed]
google_cloud_run_service.default: Creation complete after 27s [id=locations/us-central1/namespaces/dalvacationhome-427921/services/newtrial]
google_cloud_run_service.iam_member.month: Creating...
google_cloud_run_service.iam_member.month: Creation complete after 4s [id=projects/dalvacationhome-427921/locations/us-central1/services/newtrial/roles/run.invoker/allUsers]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

Outputs:
cloud_run_url = "https://newtrial-qualsimuq-us-a.run.app"
jaishankar2001@cloudshell:~ (dalvacationhome-427921) $
```

Figure 90: Terraform IaC for frontend

Deployment screenshots:

The screenshot shows the Google Cloud Run console for the 'dalvac' service in the 'us-central1' region. The service is deployed using the 'dalvac-00010-pf9' revision. The console displays various tabs including METRICS, SLOS, LOGS, REVISIONS, TRIGGERS, NETWORKING, SECURITY, INTEGRATIONS, and YAML. The 'REVISIONS' tab is active, showing a list of revisions with columns for Name, Traffic, Deployed, Revision tags, and Actions. The 'dalvac-00010-pf9' revision is selected, showing details such as CPU allocation, Startup CPU boost, Concurrency, Request timeout, Execution environment, Autoscaling, Image URL, Port, Build, Source, Command and args, CPU limit, Memory limit, Environment variables, and Volume mounts.

Name	Traffic	Deployed	Revision tags	Actions
dalvac-00010-pf9	100% (to latest)	11 minutes ago	+	⋮
dalvac-00009-66l	0%	1 hour ago		⋮
dalvac-00008-cjt	0%	1 hour ago		⋮
dalvac-00007-qg2	0%	1 hour ago		⋮
dalvac-00006-jdq	0%	17 hours ago		⋮
dalvac-00005-gv7	0%	18 hours ago		⋮
dalvac-00004-7v5	0%	18 hours ago		⋮
dalvac-00003-esn	0%	19 hours ago		⋮
dalvac-00002-vwt	0%	19 hours ago		⋮
dalvac-00001-fr	0%	20 hours ago		⋮

Figure 91: Frontend has been deployed using google cloud run

```

proj > Code > serverless_group1 > frontend > deploy.tf
1  provider "google" {
2      project = "dalvacationhome-427921" #add your project name
3      region  = "us-central1" #add your region name
4  }
5
6  resource "google_cloud_run_service" "default" {
7      name     = "newtrial" #add cloud run service name
8      location = "us-central1" #add location
9
10     template {
11         spec {
12             containers {
13                 image = "docker.io/disconnector12/dalvac:latest" #replace with yours Jaishankar
14
15                 ports {
16                     container_port = 443
17                 }
18             }
19         }
20     }
21
22     traffic {
23         percent = 100
24         latest_revision = true
25     }
26 }
27
28 resource "google_cloud_run_service_iam_member" "noauth" {
29     service = google_cloud_run_service.default.name
30     location = google_cloud_run_service.default.location
31     role     = "roles/run.invoker"
32     member   = "allUsers"
33 }
34
35 output "cloud_run_url" {
36     value = google_cloud_run_service.default.status[0].url
37 }
38

```

Figure 92: terraform script to create new deployment

```

proj > Code > serverless_group1 > frontend > Dockerfile
1  # pull official base image
2  FROM node:13.12.0-alpine
3
4  # set working directory
5  WORKDIR /frontend
6
7  # add `/app/node_modules/.bin` to $PATH
8  ENV PATH /frontend/node_modules/.bin:$PATH
9
10 # install app dependencies
11 COPY package.json ./
12 COPY package-lock.json ./
13 RUN npm install --silent
14 RUN npm install react-scripts@3.4.1 -g --silent
15
16 # add app
17 COPY . ./
18
19 # Expose the port the app runs on
20 EXPOSE 3000
21
22 # Define the environment variable for the port
23 ENV PORT 3000
24
25 # start app
26 CMD ["npm", "start"]

```

Figure 93: Dockerfile to create image

Final Architecture Diagram:

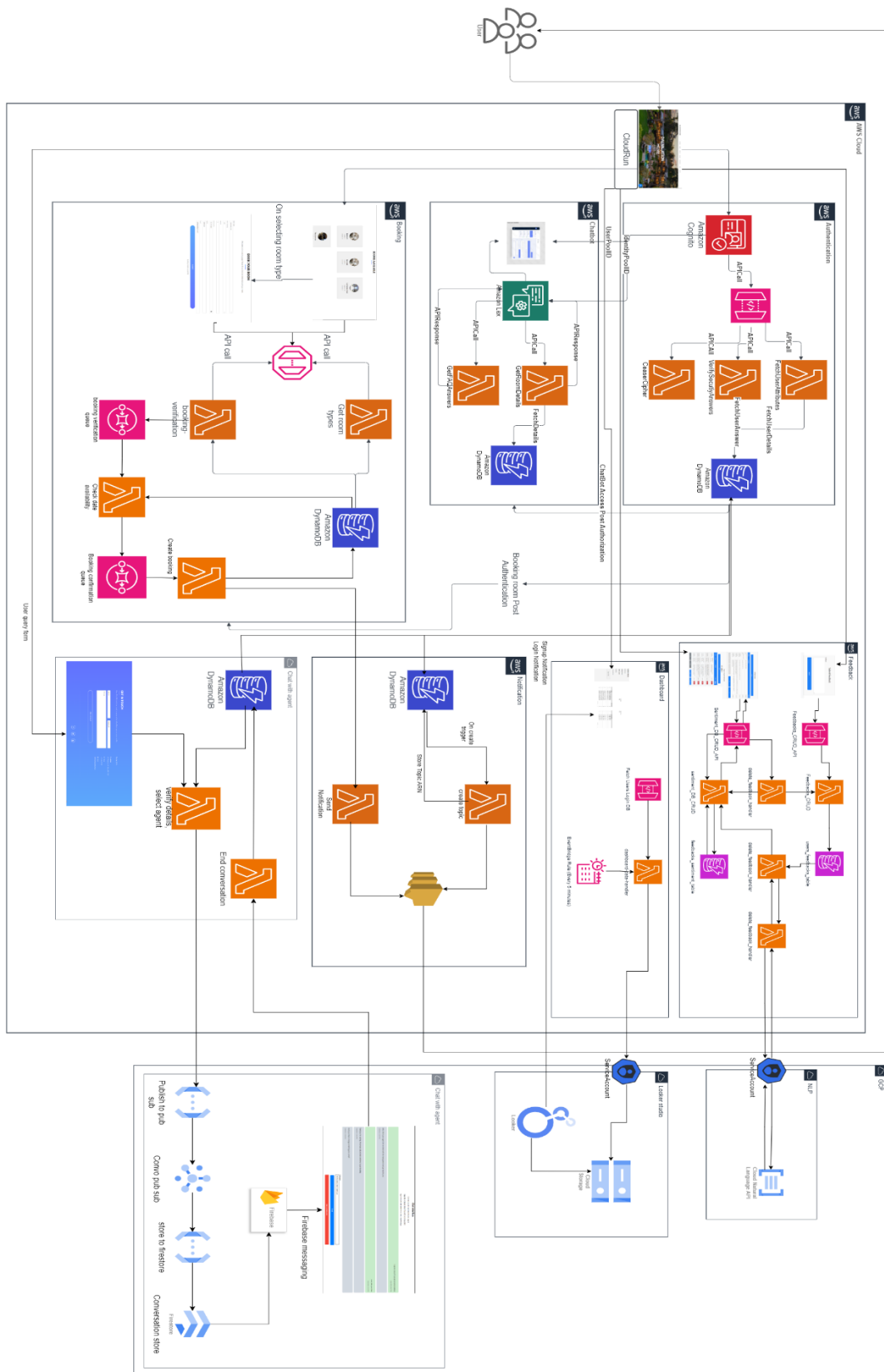


Figure 94: Final architecture diagram

Meeting Logs:

Agenda	Outcome	Link
Discussion about individual progress and further steps for integration.	Decided how to integrate the different modules with the frontend and discussion about IAC.	https://dalu-my.sharepoint.com/:v:/g/personal/ks328392_dal_ca/EUePkxoBqLVAieoSbHM9kbkB6BSnosPnl4qCB-56xK-dbQ?referrer=Teams.TEAMS-ELECTRON&referrerScenario=MeetingChicletGetLink.view.view
Creation of sprint 2 document and creation of system architecture	Created the sprint 2 document	https://dalu-my.sharepoint.com/:v:/g/personal/lv455130_dal_ca/Ebx4seKQ3_ZEmRBbWEmyEisBMxvr0gvKG3qJGV_Sb05VEQ?referrer=Teams.TEAMS-ELECTRON&referrerScenario=MeetingChicletGetLink.view.view
Complete creation of sprint 2 document and system architecture	Completed the bookkeeping tasks	https://dalu-my.sharepoint.com/:v:/g/personal/ks328392_dal_ca/EW51Jx6tIj5Pv1rofRmSWr4BaSBLYN7YOd8e2yx-pUWkXw?referrer=Teams.TEAMS-ELECTRON&referrerScenario=MeetingChicletGetLink.view.view
Meeting for knowing Updates	Knowledge on what everyone is working on	https://dalu-my.sharepoint.com/:v:/g/personal/lv455130_dal_ca/Ebx4seKQ3_ZEmRBbWEmyEisBMxvr0gvKG3qJGV_Sb05VEQ?referrer=Teams.TEAMS-ELECTRON&referrerScenario=MeetingChicletGetLink.view.view

References:

- [1] Expedia, "Help Center," *Expedia*. [Online]. Available : <https://www.expedia.ca/helpcenter/> [Accessed Jul. 04, 2024].
- [2] "CRA Services", *Canada Revenue Agency* [Online], Available: <https://www.canada.ca/en/revenue-agency/services/e-services/cra-login-services.html> [Accessed : Jun. 30, 2024].
- [3] I. Kattan, "React Landing Page Template", *Github* [Online], Available : <https://github.com/issaafalkattan/React-Landing-Page-Template> [Accessed: Jun. 28, 2024]
- [4] "Getting Started with user pools", *Amazon Web Services* [Online]. Available: <https://docs.aws.amazon.com/cognito/latest/developerguide/getting-started-user-pools.html> [Accessed : Jul. 01, 2024]
- [5] "Welcome to Hotel Halifax", *Hotel Halifax* [Online] Available: <https://www.hotelhalifax.ca/> [Accessed: Jul. 01, 2024]
- [6] A. Hitchon, "AWS_RestLambdaAPI," *GitHub* [Online], Available: https://github.com/hitchon1/AWS_RestLambdaAPI/blob/main/lambda_function.py [Accessed Jul. 2, 2024].
- [7] "Intelligent diagramming," *Lucidchart*. [Online]. Available: <https://www.lucidchart.com/pages/> [Accessed: 30-May-2024].